

6장 영역 추출과 변화 탐지, 마스크에서 결정까지

1. 이 장의 핵심 질문

□ 이 장이 끝까지 답하려는 질문은 세 가지임

1. 픽셀마다 붙인 답(마스크)을 어떻게 채점하는가. 그리고 뒤에 붙이는 필터는 무엇을 올리고 무엇을 못 올리는가
2. 마스크를 객체 목록으로 바꿔 면적·최소폭·접도 요건으로 거르는 계산은 어떤 순서로 하는가
3. 마스크의 오차에는 방향이 있다. 한쪽으로 쏠린 그 오차(편향)가 후보 목록을 어떻게 뒤집으며, 어디까지 되돌릴 수 있는가

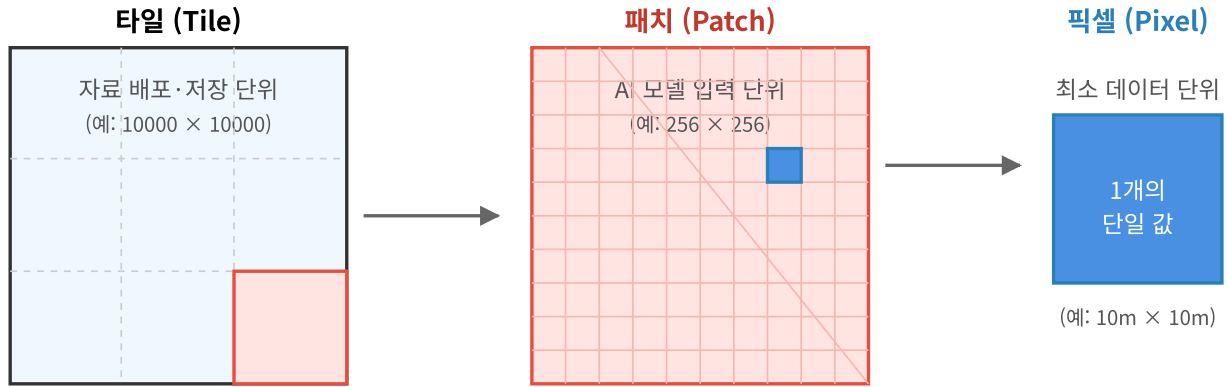
□ 5장에서 다룬 영상 분류는 "이 패치¹는 산림이다"라는 답 하나를 냄

- 이 장의 세그멘테이션은 픽셀마다 답을 붙여 "산림이 몇 헥타르이고 경계가 어디인가"에 답함
- 변화 탐지는 같은 계산을 두 시점에 걸쳐 해서 "무엇이 달라졌는가"에 답함

□ 세그멘테이션과 변화 탐지의 산출물은 **마스크**, 즉 픽셀마다 대상인지 아닌지를 적은 격자임

- 격자의 크기가 영상과 같고 칸이 서로 짝지어지므로, 영상의 칸 하나하나에 대해 대상인지 아닌지가 적혀 있음
- 모델의 출력은 처음에 픽셀마다 0~1 사이의 확률이고, 임계를 넘긴 픽셀만 대상으로 표시해 마스크를 만들(6절)
- 대상을 하나씩 구별해야 하는 과제에서는 같은 격자에 참·거짓이 아니라 덩어리 번호를 적음(7.1)
- 마스크 자체는 아직 행정이나 사업의 결정 단위가 아님. "이 칸이 대상이다"만으로는 어느 부서가 나가고 어느 땅을 보러 갈지 정해지지 않음
- 그래서 구역별 면적·객체 목록·후보표로 옮기는 계산이 뒤따름. 이 장의 6절부터 10절까지가 그 계산임

영상 분석의 공간 단위: 타일, 패치, 픽셀



타일(전체 영상) > 패치(AI 모델 입력) > 픽셀(최소 화소) 순으로 공간 포함 관계가 성립합니다.

그림 6.1 타일은 원본 영상의 배포·저장 단위이고, 패치는 모델 입력 단위이며, 픽셀은 영상의 최소 데이터 단위임

2. 도입 사례: 필터 하나가 정밀도를 0.572에서 1.000으로 올린다

- 광역지자체가 관내 보호구역의 무단 훼손을 점검하는 상황임. 6-1-change-detection-policy.log 의 실제 실행 결과로 시작함
 - 무대: 128×128 래스터(픽셀이 바둑판처럼 늘어선 격자형 영상), 픽셀 한 번 10m
 - 픽셀 하나의 면적은 $10\text{m} \times 10\text{m} = 100\text{m}^2$ 이고, 100m^2 는 0.01헥타르임(1헥타르 = 10,000 m^2)
 - 진짜 변화 픽셀 2,001개, 모델이 예측한 변화 픽셀 3,022개 — 정답보다 51% 많음. 모델이 변화를 과하게 찍었다는 뜻임
 - 필터를 걸기 전과 후를 나란히 놓으면 다음과 같음

단계	IoU	정밀도	재현율	Dice
필터 전	0.525	0.572	0.865	0.689
필터 후	0.865	1.000	0.865	0.927

- 필터는 두 규칙임 — 식생지수 차분이 0.25를 넘는 픽셀만 남기고, 연결된 덩어리가 1헥타르(100픽셀) 미만이면 지움
 - 앞 규칙은 식생이 크게 줄어든 자리만 진짜 개발로 보고, 뒤 규칙은 뭉쳐 있지 않은 작은 얼룩을 잡음으로 봄
 - 읽는 법: **정밀도**는 "변화라고 판정한 것 중 진짜의 비율"이고, **재현율**은 "진짜 변화 중 잡아낸 비율"임. 정밀도가 높으면 헛짚음이 적고, 재현율이 높으면 놓침이 적음
 - **정밀도만 0.572에서 1.000으로 오르고 재현율은 0.865에서 그대로임.** 필터가 가짜만 지우고 이미 놓친 변화는 되살리지 못했다는 뜻임. 이 비대칭이 우연이 아니라는 것을 6절에서 계산으로 확인함

- 필터를 거친 마스크를 16개 구역(4×4 격자)에 포개면 15개 구역에서 변화가 잡히고 합계 17.3헥타르가 나옴. 상위 5구역은 다음과 같음

순위	구역	변화 면적(ha)
1	구역 9	2.93
2	구역 10	2.47
3	구역 13	2.30
4	구역 6	1.78
5	구역 15	1.41

- "픽셀 3,022개가 변했다"는 담당 부서를 정하지 못하지만, "구역 9에서 2.93헥타르가 개발되었고 관내 최대"는 점검반의 방문 순서를 정함
- 이 장은 이 두 계산 — **채점해서 걸러 내기, 구역별로 모아 순서를 매기기** — 을 어떤 절차로 하는지, 그리고 대상이 픽셀이 아니라 개별 필지일 때 무엇이 달라지는지를 다룸

3. 무엇을 만들 것인가 정하기

- 계산을 시작하기 전에 "어떤 답을 낼 것인가"를 먼저 정함

3.1 과업 유형

- 주택가 사진 한 장을 놓고도 답의 층위가 갈림

과업	한 입력에 대한 답	답할 수 있는 질문
시맨틱 세그멘테이션	픽셀마다 클래스 1개	산림이 몇 헥타르이고 경계가 어디인가
인스턴스 세그멘테이션	객체마다 마스크 1개	각 건물의 면적과 형상은 얼마인가
파놉틱 세그멘테이션	픽셀당 클래스 + 객체 식별자	객체와 배경을 한 지도에서 같이 다룰 수 있는가
변화 탐지	픽셀당 변화 여부	두 시점 사이 어디가 달라졌는가

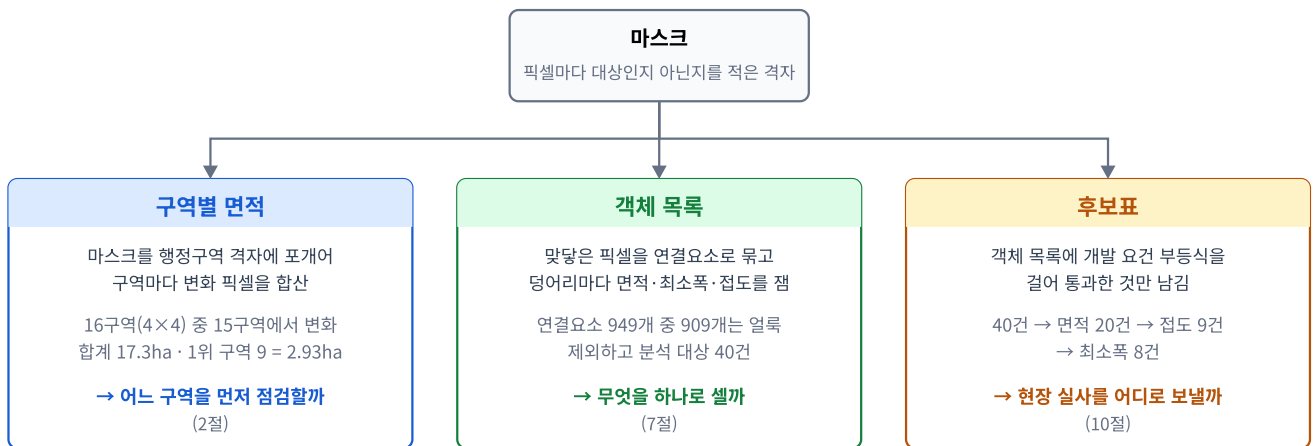
- 넷의 차이는 **객체를 구분하는지**에 있음. 시맨틱은 칸마다 종류만 붙여 숲 1과 숲 2를 구분하지 않고, 인스턴스는 객체마다 따로 마스크를 주며, 파놉틱은 둘을 합쳐 종류와 객체 번호를 같이 가짐. 변화 탐지는 종류가 아니라 "바뀌었는가"만 답함
- 참고 링크: <https://rubber-tree.tistory.com/134>
- **변화 탐지는 별도 계열이 아니라 두 시점을 입력으로 받는 세그멘테이션임.** 계산의 뼈대는 같고 달라지는 지점이 둘 있음

1. 두 시점이 같은 좌표 격자에 포개지지 않으면 경계선을 따라 가짜 변화가 생기므로, 두 영상을 같은 격자에 포개는 **정합**이 0단계가 됨(4.2)
2. 검증 지표의 우선순위가 반대로 놓일 수 있음. 재난 피해 평가에서는 농친 피해가 비용이 크고, 단속 행정에서는 잘못 잡은 변화가 비용이 큼(6.2)

3.2 산출 단위를 착수 시점에 정한다

- 픽셀 마스크에서 끝낼지, 객체 목록까지 갈지, 구역 집계까지 갈지를 미리 정함
 - 이유: 필요한 라벨 형식과 이후 계산이 완전히 달라지기 때문임
 - 끝에 가서 "사실 건물마다 면적이 필요했다"고 하면 라벨부터 다시 만들어야 함

마스크를 결정 단위로 옮기는 세 경로



셋 모두 픽셀을 세어 사람이 세는 단위로 바꾸는 계산임. 어느 경로를 쓸지 착수 시점에 정함(3.2)

그림 6.2 마스크를 결정 단위로 옮기는 세 경로. 구역별 면적은 구역마다 변화 픽셀을 합산해 점검 순서를 내고, 객체 목록은 연결요소로 묶어 덩어리마다 면적·최소폭·접도를 재며, 후보표는 그 목록에 개발 요건을 걸어 실사 대상을 남김. 상자 안의 수치는 2절·7절·10절의 실행 결과임

산출 단위	필요한 층위	주의할 점
픽셀(면적 합계)	시맨틱만으로 충분	픽셀 크기에서 면적으로 환산
객체(면적·형상)	인스턴스, 또는 시맨틱 + 연결요소	맞닿은 객체가 하나로 묶임
구역(집계표·우선순위)	시맨틱으로 충분	구역 경계를 무엇으로 잡느냐가 순위를 바꿈

- 객체가 조밀하게 붙은 도심에서는 "시맨틱 마스크 + 연결요소"로 인스턴스를 대신하는 방법이 성립하지 않음. 맞닿은 두 건물의 1 픽셀이 서로 이어져 있어 한 덩어리로 묶여 버림
- 2절의 사례는 산출 단위가 구역이었으므로 객체 단위 계산이 필요 없었음. 10절의 부지 탐색은 산출 단위가 객체이므로 7·8절의 계산이 전부 필요함

3.3 마스크로 답할 수 없는 질문도 있다

- 세그멘테이션이 재는 것은 면적과 형상, 즉 "무엇이 거기 있는가"임. "그것이 얼마의 값을 갖는가"의 정보가 아님
 - 영상으로 부동산 가치를 예측하려는 시도가 있었음. 다만 쓰인 것은 건물 윤곽의 면적·형상이 아니라 신경망이 학습한 동네의 외관 인상이고, 윤곽 기하로 가치를 예측하는 계열은 확인되지 않음
 - 런던 거래 11만 건을 쓴 연구²가 **설명력**(R^2 , 집값의 차이를 몇 %나 맞췄는가)으로 비교한 결과임

무엇을 넣었나	설명력	한 개 구를 통째로 빼고 학습
주택 속성만(면적·방 수·건물나이 등)	74.9%	69.0%
스트리트뷰 사진만	4.9%	2.7%
항공영상만	15.2%	6.2%
속성 + 항공영상	80.9%	75.9%

- 이 표에서 읽을 것
 - **영상 단독으로는 가치를 거의 설명하지 못함** — 스트리트뷰만으로는 4.9%뿐임. 영상은 속성 모형에 없는 보조 변수로만 쓰임
 - 그 비교 기준에는 인근 최근 실거래가도, 가까운 집끼리 값이 비슷하다는 성질(공간 자기상관)도 들어 있지 않음. 실제 감정에서 가격을 지배하는 신호가 그 둘이므로, **영상이 올린 뒤편은 공간 모형이 직접 잡았을 위치 효과를 대신 잡은 것일 수 있음**
 - 오른쪽 열이 영상의 쓸모를 보여 줌. 한 개 구를 통째로 빼고 학습하면 — 즉 **그 동네의 거래 자료가 하나도 없으면** — 69.0%에서 75.9%로 오름. 영상이 하는 일은 정확도를 더 높이는 것이 아니라 **자료가 없는 곳을 메우는 것임**
 - 그래서 실거래가 촘촘히 쌓이는 시장에서는 쓸 자리가 거의 없음. 한국부동산원의 자동산정모형(AVM, 실거래·공시 자료로 값을 자동 산정하는 프로그램)도 영상을 쓰지 않음
 - 같은 면적의 건물도 두 블록 떨어지면 값이 갈림. 그 차이는 학군·임차인 구성·권리관계처럼 영상에 기록되지 않는 조건에서 옴
 - 그래서 이 장은 영상 피쳐로 가치를 예측하는 예제를 두지 않고, **마스크에서 바로 계산되는 질문**을 다룸 — 개발할 수 있는 땅이 어디 남았는가
 - 면적·최소폭·접도 길이는 모두 마스크에서 직접 나오고, 개발 요건은 이 세 값에 대한 부등식으로 적힘. 10절에서 쓰는 요건이 "면적 $\geq 1,000\text{m}^2$ 그리고 접도 $\geq 12\text{m}$ 그리고 최소폭 $\geq 15\text{m}$ "임

4. 입력 자료가 조건을 통과하는지 확인하기

- 모델을 돌리기 전에 "이 영상으로 그 대상을 쥔 수 있는가"부터 확인함

4.1 대상이 몇 픽셀인가

□ 픽셀 크기 g 와 대상의 대표 폭 w 에서 대상의 픽셀 폭은 w/g 로 계산함

- 폭 12m 주택은 10m 격자에서 $12 \div 10 = 1.2$ 픽셀, 0.5m 항공영상에서 $12 \div 0.5 = 24$ 픽셀임. 같은 대상이지만 다른 과제임
- 1.2픽셀 대상은 경계 오차 한 겹이 대상 전체를 지우거나 몇 배로 키움. 통째로 대상 안에 들어가는 칸이 하나도 없어 경계선이 모든 칸을 가로지르므로, 그 칸들을 배경으로 보느냐 대상으로 보느냐에 따라 0칸에서 4칸까지 갈림

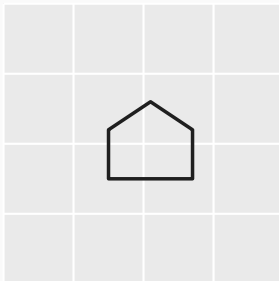
격자가 정하는 오차의 하한 — 경계선이 가로지른 칸은 어떤 규칙으로도 갈린다

같은 주택(폭 12m, 108㎡)을 두 해상도로 관측함 · 검은 선이 주택의 실제 경계선, 주황은 그 선이 가로지른 칸

① 10m 격자 — 주택 폭이 1.2픽셀

통째로 주택 안에 들어가는 칸이 하나도 없음. 모든 칸을 경계선이 가로지름

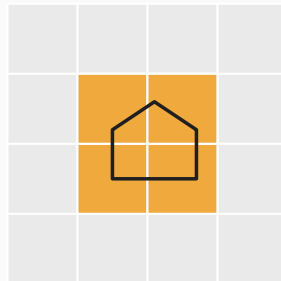
경계선이 지난 칸을 배경으로 봄



0칸

침식 — 가장 작게 잡은 값

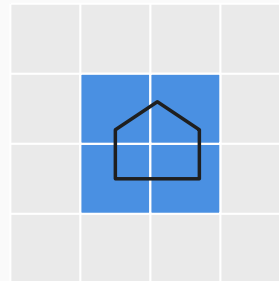
주택 경계선과 격자



안쪽 0칸 + 경계 4칸

경계선이 가로지른 칸이 주황

경계선이 지난 칸을 대상으로 봄



4칸

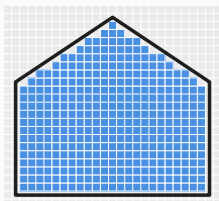
팽창 — 가장 크게 잡은 값

판정 규칙 하나로 0칸과 4칸 사이를 오감. 대상이 통째로 사라지거나 몇 배로 커짐

② 0.5m 격자 — 주택 폭이 24픽셀

안쪽은 어떤 규칙으로도 대상이고, 경계선이 지난 테두리 한 줄만 갈림

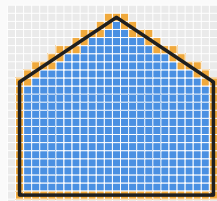
경계선이 지난 칸을 배경으로 봄



387칸

침식 — 가장 작게 잡은 값

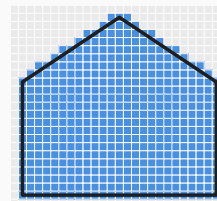
주택 경계선과 격자



안쪽 387칸 + 경계 92칸

경계선이 가로지른 칸이 주황

경계선이 지난 칸을 대상으로 봄



479칸

팽창 — 가장 크게 잡은 값

판정이 갈려도 387칸과 479칸 사이이고, 지붕과 몸통의 모양은 그대로 남음

같은 주택인데 1.2픽셀에서는 0칸~4칸으로 갈리고, 24픽셀에서는 387칸~479칸에 머무름

격자가 애매하게 만드는 것은 경계에 수직으로 한 겹뿐임. 다만 ①처럼 폭이 2픽셀 미만이면 그 한 겹이 전부라 통째로 사라짐
모델이 경계를 흐리게 그리면 큰 대상에서도 여러 겹이 짝임 — 이 장의 실습 자료는 32필지 중 8필지가 두 겹 침식 (5월·8.2)

■ 통째로 주택 안

■ 경계선이 가로지름

□ 면적 오차의 하한도 이 계산에서 나옴. 경계 한 겹의 오차가 바꾸는 픽셀 수는 대략 둘레만큼이므로, 상대 오차는 둘레 ÷ 면적임

- 왜 둘레만큼인가: 테두리를 한 겹 깎으면 사라지는 칸은 경계를 따라 한 줄로 늘어선 칸들이고, 그 줄의 길이가 곧 둘레임
- 정사각형이면 둘레는 $4s$, 면적은 s^2 이므로 $4s \div s^2 = 4/s$ 이고, **한 변의 길이만으로 정해짐**
- 한 변 100m면 $4 \div 100 = 4\%$, 한 변 32m면 $4 \div 32 = 12.5\%$
- 면적을 5% 이내로 재야 한다면 $4/s \leq 0.05$ 에서 $s \geq 80$ 이므로, 대상의 한 변이 픽셀 크기의 80배 이상이어야 함. 이 계산을 착수 시점에 해 두면 해상도가 부족한 자료로 시작했다가 끝에 가서 면적 오차를 발견하는 일을 피함

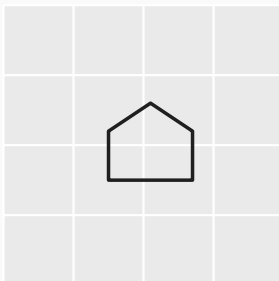
경계선이 가로지른 칸은 둘레만큼 생기므로, 작은 대상일수록 그 몫이 커진다

주택 = 몸통 $12m \times 7m$ + 지붕 $12m \times 4m = 108m^2$, 둘레 $40.4m$ · 검은 선이 주택의 실제 경계선

① 10m 격자 — 주택 폭이 1.2픽셀

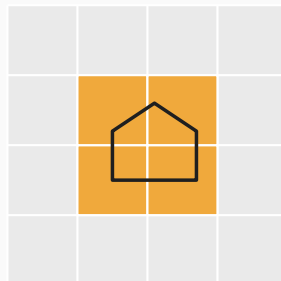
경계선이 모든 칸을 가로지르므로 경계 칸이 곧 전체임

1. 경계선을 격자 위에 얹음



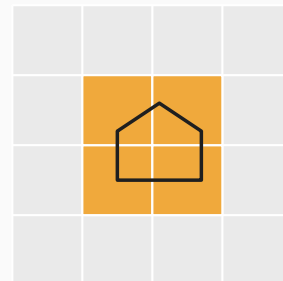
선이 칸을 가로지름

2. 그 선이 지난 칸을 표시



경계 4칸

3. 경계 칸이 차지하는 몫



$4 \div 4 = 100.0\%$

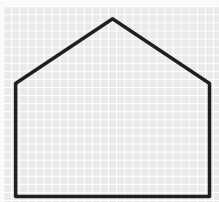
경계 칸 4칸 $\approx$ 둘레 $40.4m \div 10m = 4$ 칸 — 경계 칸 수는 대략 둘레만큼임

판정에 따라 흔들리는 몫 = $4 \div 4 = 100.0\%$ (면적 $108m^2$ 는 10m 칸으로 1칸 값어치뿐인데 4칸에 걸침)

② 0.5m 격자 — 주택 폭이 24픽셀

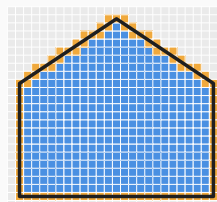
경계선이 테두리 한 줄만 가로지르므로 안쪽은 흔들리지 않음

1. 경계선을 격자 위에 얹음



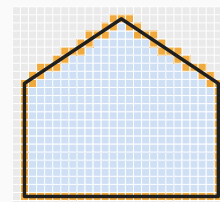
선이 칸을 가로지름

2. 그 선이 지난 칸을 표시



경계 92칸

3. 경계 칸이 차지하는 몫



$92 \div 479 = 19.2\%$

경계 칸 92칸 $\approx$ 둘레 $40.4m \div 0.5m = 81$ 칸 — 경계 칸 수는 대략 둘레만큼임

판정에 따라 흔들리는 몫 = $92 \div 479 = 19.2\%$

한 변이 10배 길어지면 면적은 100배가 되지만 둘레는 10배에 그치므로, 경계가 차지하는 몫은 1/10로 줄어듦

격자가 애매하게 만드는 것은 경계에 수직으로 한 겹뿐임. 다만 ①처럼 폭이 2픽셀 미만이면 그 한 겹이 전부라 통째로 사라짐
모델이 경계를 흐리게 그리면 큰 대상에서도 여러 겹이 깎임 — 이 장의 실습 자료는 32필지 중 8필지가 두 겹 침식 (5월·8.2)

■ 통째로 주택 안

■ 경계선이 가로지름

그림 6.3 대상의 픽셀 수에 따른 경계 1픽셀 오차의 상대적 영향 비교

픽셀 크기	대상	픽셀 폭	판정
10m	주택 12m	1.2	개별 계측 불가, 구역 집계만 가능
10m	개발 패치 200m	20	구역 면적 집계에 사용 가능
1m	필지 34m	34	개별 계측 가능, 면적 편향 보정 필요
0.5m	주택 12m	24	객체 검출 가능, 경계 정밀도 확인 필요

- 이 4/s 관계는 8절에서 다시 나옴. 같은 침식 깊이라도 작은 필지가 상대적으로 더 많이 깎이는 이유가 여기에 있음

쉽게 말하면. 액자 테두리의 폭이 같아도 작은 그림이 더 많이 가려짐. 테두리가 먹는 넓이는 그림의 둘레에 비례하는데, 그림의 넓이는 한 변의 제곱으로 커지기 때문임.

4.2 두 시점이 같은 격자에 포개져 있는가

- 위성은 촬영마다 궤도와 자세가 미세하게 달라, **기하보정**(영상을 좌표계에 맞춰 얹히는 처리) 후에도 두 시점이 몇 픽셀 어긋날 수 있음
 - 어긋난 채 차분하면 — 즉 같은 자리의 두 값을 그대로 빼면 — 건물 가장자리·도로변·하천변을 따라 값이 크게 달라지고 이 픽셀들이 변화로 판정됨
 - 건물은 그대로인데 지붕 경계가 한 픽셀 밀려 변화로 잡히는 형태임. 밀린 칸에서는 한쪽 시점은 지붕 값, 다른 시점은 바닥 값이 들어오므로 차이가 커짐
 - 변화 탐지의 0단계는 두 시점을 같은 좌표 격자에 포개는 **시점 정합(co-registration)**이고, 2장에서 다룬 좌표계와 정합 품질이 여기서 판정 기준이 됨

주의. 시점 정합 오차는 지표를 정상으로 보이게 함. 두 시점이 한 픽셀 어긋나면 모든 경계를 따라 가짜 변화 띠가 생기는데, 이 띠는 면적이 작아 전체 IoU를 크게 떨어뜨리지 않음. 변화 마스크가 선형 띠 모양으로 나타나는지 육안으로 확인하고, 어긋남이 의심되면 정합을 다시 한 뒤 지표를 읽음.

4.3 거짓 변화를 거르는 규칙

- 정합을 맞춰도 지표가 변하지 않았는데 값이 달라지는 경우가 남음. 구름과 그림자가 지표를 가리면 밝기가 급변하고, 여름 녹음과 겨울 나지는 같은 산림인데 값이 다름. 이런 값 변화를 **거짓 변화(false change)**라 함
 - 대응은 순서대로 씀. 앞의 규칙이 뒤의 규칙이 다룰 자료를 정리해 주기 때문임
 - 구름을 먼저 지우지 않고 식생 차이를 보면, 구름에 가려 식생지수가 떨어진 자리를 개발로 판정하게 됨

순서	대응	제거하는 원인
1	대기보정 영상 사용	대기 상태 차이
2	구름·그림자 마스크	구름과 그림자
3	지수 차분 임계	계절 변동, 관측 잡음
4	최소 면적 필터	점점이 흩어진 오탐

- **대기보정**은 센서에 들어온 값에서 대기가 흐트리고 흡수한 몫을 걷어 내 지표 자체의 반사율로 바꾸는 처리임. 보정하지 않으면 같은 땅도 촬영일의 대기 상태에 따라 값이 달라짐
- 2절의 실습은 3·4단계만 적용한 결과였음
- 두 값은 자료의 성질에서 나옴. 식생지수 차분 임계 0.25는 진짜 개발이 식생지수를 크게 떨어뜨린다는 성질에서, 최소 면적 1헥타르는 개발이 덩어리로 일어나고 잡음은 흩어진다는 성질에서 정한 값임
- 두 값을 정한 뒤에는 6.3처럼 필터 전후의 정밀도·재현율로 효과를 측정해 확정함. "이 정도면 되겠지"로 끝내지 않음

5. 마스크는 어디서 오는가

- 실습에서는 마스크를 미리 만들어 둔 자료에서 불러오지만, 그 마스크가 어떤 계산에서 나오는지 알아야 오차의 방향(8절)을 이해할 수 있음
- **문제 상황**. 10m 격자 위성영상에서 관내 산림 면적을 헥타르 단위로 집계해야 함. 5장의 패치 분류는 패치 한 장에 답 하나만 내므로, 패치 안에 산림과 나지가 섞여 있으면 비율을 알 수 없음. 경계가 어느 픽셀인지 모르면 면적을 셀 수 없음
 - 픽셀마다 답을 내려면 출력의 크기가 입력과 같아야 함. 입력이 256×256 이면 답도 256×256 이어야 어느 칸이 산림인지 적을 수 있음
 - 그런데 5장에서 본 합성곱 신경망은 **풀링을 지날 때마다 해상도를 절반으로 줄임**. 그대로 두면 출력이 입력보다 작아짐
- 먼저 두 연산의 이름을 정리함
 - **풀링(pooling)**은 2×2 칸 네 개를 한 칸으로 줄이는 연산임. 네 값 중 최댓값을 남기는 방식이 흔함. 한 번 지날 때마다 가로·세로가 각각 절반이 되므로 칸 수는 $1/4$ 이 됨
 - **업샘플(upsample)**은 그 반대로 한 칸을 2×2 로 되돌려 가로·세로를 각각 두 배로 늘리는 연산임
 - 풀링이 네 칸을 한 칸으로 줄이면 "네 칸 중 어느 칸이었나"는 사라지고, 업샘플로 칸 수를 되돌려도 그 구분은 돌아오지 않음
- U-Net(Ronneberger 외, 2015)은 이 문제를 인코더-디코더 두 부분으로 나눠 풀
 - **인코더**는 해상도를 줄이면서(풀링) 의미를 압축함. "이 구역은 산림이다" 같은 넓은 범위의 판단은 남고, 칸 단위의 위치는 흐려짐
 - **디코더**는 해상도를 되돌리면서(업샘플) 클래스 지도를 복원함

- 인코더의 얇은 층 특징을 같은 해상도의 디코더 층에 그대로 이어 붙이는 통로가 **스킵 연결(skip connection)**임. 줄어들기 전의 위치 정보를 디코더로 곧장 넘겨 주는 길임
- 스킵 연결이 왜 필요한지는 그것을 끊었을 때 남는 값으로 확인함
 - 풀링을 세 번 지나면 $2 \times 2 \times 2 = 8$ 이므로, 가장 깊은 층의 특징 한 화소가 담당하는 지상 영역이 8×8 픽셀이 됨
 - 10m 격자에서 8픽셀은 80m이므로 그 영역은 $80m \times 80m$ 이고, **이보다 작은 경계는 위치를 특정할 수 없음**. 한 화소가 80m 사방을 뭉쳐 대표하므로, 그 안에서 경계가 어디를 지나는지 그 화소만 보고 알 수 없다는 뜻임
 - 업샘플은 해상도를 되돌리지만 줄이는 과정에서 버린 세부를 복원하지는 않음. 칸 수만 늘어남
 - 스킵 연결이 얇은 층의 위치 정보를 다시 공급해 이 하한을 끌어내림. 얇은 층은 아직 풀링을 적게 지났으므로 경계의 위치를 더 잘게 갖고 있음

U-Net: 줄였다가 키우고, 잃어버린 위치는 옆에서 받아 온다

왼쪽 절반이 인코더(해상도를 줄이며 의미를 얻는다), 오른쪽 절반이 디코더(해상도를 키우며 경계를 복원한다)

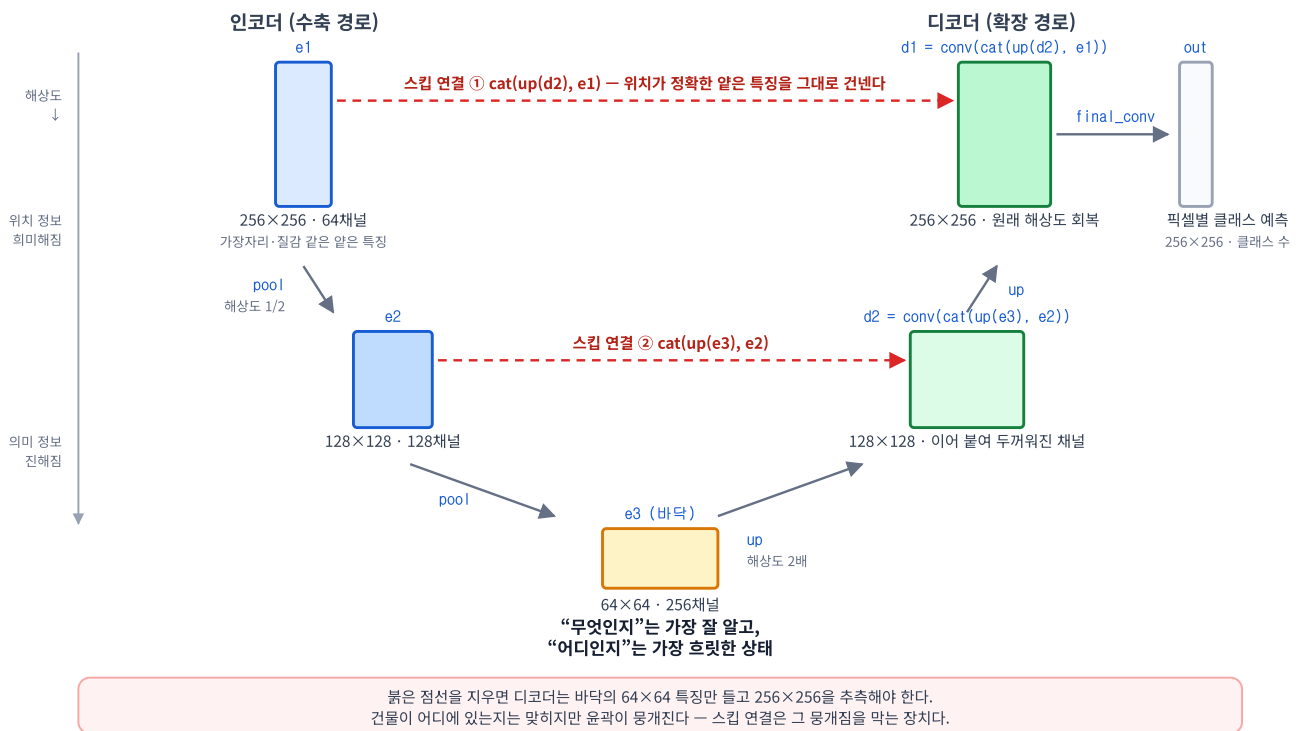


그림 6.4 U-Net의 구조. 왼쪽 아래로 내려가는 인코더는 pool 을 지날 때마다 해상도가 절반이 되고, 오른쪽 위로 올라가는 디코더는 up 으로 해상도를 되돌림. 아래로 갈수록 상자가 납작하고 두꺼워지는 것은 공간 해상도가 줄고 채널 수가 늘는 상태임. 붉은 점선 두 개가 스킵 연결이며, 같은 해상도였던 인코더 특징을 건너편 디코더에 그대로 이어 붙임

- 이 구조가 이 장의 오차와 이어짐. 해상도를 줄였다 되돌리는 과정에서 경계의 위치 정보가 흐려짐
 - 경계 칸은 대상과 배경이 섞여 있어 판정이 애매한 칸임. 애매한 칸에서 모델이 배경 쪽으로 기울면 그 칸은 0이 됨
 - 그 결과 예측 마스크의 가장자리가 실제보다 안쪽으로 한 겹 깎임. 이것이 8절에서 다룰 **경계 침식**이며, 이 장에서 가장 큰 영향을 미치는 오차임

- 방향이 한쪽인 것이 중요함. 애매한 칸이 배경으로 기울면 면적은 **항상** 작게 나옴. 커지는 쪽으로는 틀리지 않음
- 6절부터 10절까지는 설명과 실습을 나누지 않음 — 각 절 첫머리에 실행할 명령이 나오고 그 아래에 코드와 실제 실행 결과가 이어짐. 순서대로 따라가면 이 장의 모든 수치를 직접 만들게 됨
- 먼저 패키지를 설치함. 기본 실습 코드가 쓰는 것은 NumPy·pandas·SciPy·Matplotlib뿐이라 딥러닝 패키지는 필요 없고, GPU 없이 노트북에서 끝까지 돌아감. GPU가 있으면 선택 실습 6-0a 에서 작은 U-Net을 직접 학습할 수 있으며, 이 경우에만 PyTorch가 필요함

```
cd geoai_public # 저장소 루트 - 이 장의 명령은 모두 여기서 실행함
pip install -r lecture_practice/requirements-student.txt
```

- (선택) GPU U-Net 실습: PyTorch를 설치하고 장치를 확인한 뒤 6-0a 를 실행함. Apple Silicon Mac은 cuda 를 mps 로, 장치 자동 선택은 auto , GPU가 없으면 cpu 로 구조만 확인할 수 있음. 지정한 cuda · mps 를 쓸 수 없으면 CPU로 전환하지 않고 중단함. 기존 마스크와 기본 실습 결과는 덮어쓰지 않음

```
python lecture_practice/setup_torch.py
python lecture_practice/check_env.py
python lecture_practice/chapter6/code/6-0a-unet-change-detection.py --device cuda
```

- 이 장에서 기본으로 실행할 코드는 두 개이며 순서가 정해져 있음

순서	실행할 파일	하는 일	절
1	6-1-change-detection-policy.py	사전 생성 변화 마스크 채점·거짓 변화 필터링·구역 집계	6, 9.1
2	6-2-site-sourcing.py	객체화·기하 피쳐·요건 필터·편향 보정·대조군·절단선	7~10

- 1번은 저장소에 포함된 change_masks.npz 를 바로 읽고, 2번은 저장소에 포함된 urban_block.npz 를 입력으로 받음. 분석 1과 분석 2는 서로 독립임
- .npz 는 NumPy 배열 여러 개를 이름을 붙여 한 파일에 묶어 둔 형식임. 여기서는 두 시점 영상·정답 마스크·예측 마스크가 한 파일에 들어 있음
- 두 코드 모두 인터넷 없이 돌아가며 GPU도 쓰지 않음(GPU는 선택 실습 6-0a 에서만 씀)
- 실제 모델(U-Net 같은)을 학습하는 대신, **그 모델이 냈을 법한 출력(변화 마스크·부지 마스크)을 미리 만들어 둔 자료에서 불러옴**. 이 장의 계산 대상은 모델 내부가 아니라 마스크를 채점·필터·집계·보정하는 절차임
- 본문에 인용한 출력은 모두 lecture_practice/chapter6/results/ 의 실행 로그에서 그대로 가져온 것이며, 어느 파일에서 왔는지를 출력 블록 첫 줄에 적어 두었음 — 화면에 나온 값과 그 파일을 한 줄씩 대조하면 됨

6. 마스크를 채점하기

- 검증은 두 층위로 나눔
 - 픽셀 단위 채점 — 마스크가 정답과 얼마나 겹치는가
 - 객체 단위 채점 — 객체를 몇 개 찾았고 각 객체의 면적을 얼마나 정확히 잰는가
 - 어느 층위를 쓸지는 3.2에서 정한 산출 단위가 정함. 구역 면적 집계는 최종 산출물이면 픽셀 단위로 충분하고, 객체 목록이 판단 단위이면 픽셀 지표만으로는 부족함. 이 절은 픽셀 단위를 다루고, 객체 단위는 8절에서 다룸
- 저장소에 포함된 두 시점 NDVI 래스터와 변화 마스크를 읽어 채점·필터·집계함
 - **NDVI**는 식생지수로, 식생이 짙을수록 높아지고 -1에서 1 사이 값을 가짐. 이 자료에서 개발 전 식생은 0.65대이고 개발 후 나지는 0.15대임
 - 6절과 9.1의 숫자는 아래 명령의 출력임

```
python lecture_practice/chapter6/code/6-1-change-detection-policy.py
```

6.1 혼동행렬과 픽셀 단위 지표

- 픽셀 단위의 평가는 모든 픽셀을 다음 4가지 상태로 분류하는 ****혼동행렬(Confusion Matrix)****에서 출발함

	예측: 변화(양성)	예측: 미변화(음성)
실제: 변화(양성)	참양성 TP — 잡았다	거짓음성 FN — 놓쳤다
실제: 미변화(음성)	거짓양성 FP — 헛짚었다	참음성 TN — 맞게 넘겼다

- 네 칸을 읽는 순서는 "실제가 무엇이었나"를 먼저 보고 "예측이 그것을 맞았나"를 보는 것임. 이름의 앞부분(참·거짓)은 예측이 맞았는지를, 뒷부분(양성·음성)은 예측이 무엇이라고 답했는지를 뜻함

```
# 주석에 붙인 출력의 출처: lecture_practice/chapter6/results/6-1-change-detection-policy.log
def iou_prf(pred, true):
    """예측·진짜 변화 마스크의 IoU·정밀도·재현율·Dice."""
    pred, true = pred.astype(bool), true.astype(bool)
    inter = np.logical_and(pred, true).sum()          # TP (잡았다)
    union = np.logical_or(pred, true).sum()           # TP + FP + FN
    iou = inter / union if union else 0.0
    prec = inter / pred.sum() if pred.sum() else 0.0  # 정밀도: TP / (TP + FP)
    rec = inter / true.sum() if true.sum() else 0.0   # 재현율: TP / (TP + FN)
    dice = 2 * inter / (pred.sum() + true.sum()) if (pred.sum() + true.sum()) else 0.0
    return iou, prec, rec, dice

# 필터를 걸기 전의 예측 마스크를 그대로 채점함
iou0, p0, r0, d0 = iou_prf(pred, true_change)
# 출력: [6.2 평가] 거짓변화 필터링 전 (예측 vs 진짜)
#      IoU 0.525 | 정밀도 0.572 | 재현율 0.865 | Dice 0.689
#      → 정밀도가 낮다 = 거짓양성(FP, 구름·계절 잡음)이 섞여 있다.
```

□ 코드가 하는 일을 줄 단위로 보면

코드	하는 일	세면
<code>astype(bool)</code>	0.1 격자를 참·거짓 격자로 바꿈. 아래 논리 연산을 쓰기 위한 준비	—
<code>np.logical_and(pred, true)</code>	둘 다 참인 칸만 남김	TP
<code>np.logical_or(pred, true)</code>	둘 중 하나라도 참인 칸을 남김(둘 다 0인 칸만 빠짐)	TP + FP + FN
<code>pred.sum()</code>	예측이 변화라고 답한 칸 수	TP + FP
<code>true.sum()</code>	진짜 변화 칸 수	TP + FN
<code>if pred.sum() else 0.0</code>	예측이 하나도 없을 때 0으로 나누는 것을 막음	—

□ 네 지표의 식을 나란히 적으면

정밀도 = $TP / (TP + FP)$

예측이 변화라고 한 것 중 진짜의 비율

재현율 = $TP / (TP + FN)$

진짜 변화 중 잡아낸 비율

IoU = $TP / (TP + FP + FN)$

겹친 칸 ÷ 둘을 합친 칸

Dice = $2TP / ((TP + FP) + (TP + FN))$ 겹친 칸을 두 번 세고 두 마스크의 칸 수 합으로 나눔

○ TN은 네 식에 하나도 들어가지 않음. 변화가 아닌 칸을 변화가 아니라고 맞힌 것은 세지 않는다는 뜻이고, 그 이유는 6.4에서 다름

□ 숫자를 넣어 보는 예 — 정답과 예측이 각각 100픽셀이고 50픽셀이 겹치는 경우

○ 혼동행렬로 분해하면 $TP = 50$ (겹친 칸), $FP = 100 - 50 = 50$ (예측에만 있는 칸), $FN = 100 - 50 = 50$ (정답에만 있는 칸)임

○ 정밀도 = 재현율 = $50 \div (50 + 50) = 50 \div 100 = 0.5$ (두 식의 분모가 같은 경우임)

○ IoU = $50 \div (50 + 50 + 50) = 50 \div 150 = 0.333$

○ Dice = $(2 \times 50) \div (100 + 100) = 100 \div 200 = 0.5$

○ 정답의 절반을 맞혔는데 IoU가 0.5가 아니라 0.333인 이유는 분모가 두 마스크의 어긋난 부분(FP 50과 FN 50)을 모두 더하기 때문임. 분자는 50인데 분모는 150이 됨

○ Dice는 같은 어긋남을 한 번만 세는 효과가 있어 같은 상황에서 IoU보다 큼. 두 값은 같은 지표끼리만 비교함

○ **IoU 0.5는 절반을 맞힌 결과가 아니라 상당히 겹친 결과임**

□ 실행 결과를 혼동행렬의 관점으로 읽을 것

○ 재현율 0.865는 높는데 정밀도가 0.572로 낮음 — 진짜 변화는 대부분 잡았으나(FN이 적음), 그 판정에 가짜(FP)가 많이 섞였다는 뜻임

○ IoU 0.525는 정밀도와 재현율을 함께 반영한 값이라 둘 중 낮은 쪽에 끌려 내려감. 분모에 FP와 FN이 같이 들어가므로 어느 한쪽만 커도 값이 떨어짐

6.2 정밀도와 재현율은 오류의 방향을 가른다

□ 두 지표가 재는 것이 다름 (혼동행렬 기준)

○ 정밀도가 낮다 = 예측한 것 중 가짜(FP)가 많다 (헛짚음) → 현장에 나갔는데 아무 일도 없음

○ 재현율이 낮다 = 진짜 변화 중 놓친 것(FN)이 많다 (놓침) → 훼손이 진행되는데 점검 대상에 오르지 않음

○ 어느 쪽을 우선할지는 결정의 성격이 정함. 재난 피해 평가에서는 놓친 피해(FN)가 구호로 이어지지 않으므로 재현율이 우선하고, 판정이 제재로 이어지는 단속 행정에서는 잘못 잡은 변화(FP)가 행정력 낭비를 만들므로 정밀도가 우선함

○ 판정 임계값이 두 값의 균형을 움직이는 손잡이임. 모델은 칸마다 "변화일 확률"을 내고, 그 확률이 임계를 넘으면 1로 찍음

○ 임계를 낮추면 더 많이 변화라고 예측하므로 FP가 늘고 FN이 줄어, 재현율이 오르고 정밀도가 내려감. 임계를 높이면 그 반대임

쉽게 말하면. 임계는 그물의 눈 크기임. 성기게 치면 많이 건지지만 쓰레기도 들어오고, 촘촘히 치면 쓰레기는 줄지만 작은 고기도 빠져나감.

6.3 필터가 올리는 것과 못 올리는 것

- 4.3의 마스킹 규칙과 최소 면적 필터는 **예측 안의 가짜(FP)를 깎는 장치**임. 그래서 정밀도만 올리고 재현율은 올리지 못함
 - 같은 실행이 이어서 3-4단계 필터를 걸고 다시 채점함

```
# 주석에 붙인 출력의 출처: lecture_practice/chapter6/results/6-1-change-detection-policy.log
```

```
from scipy import ndimage
```

```
# [1] NDVI 차분 임계 - 진짜 토지변화는 식생지수를 크게 떨어뜨림
```

```
dndvi = np.abs(ndvi_t2 - ndvi_t1)
```

```
ndvi_ok = dndvi > 0.25
```

```
pred_f1 = np.logical_and(pred, ndvi_ok)
```

```
# [2] 최소 변화 면적 - 연결요소로 묶어 1ha(100px) 미만 얼룩을 지움
```

```
min_px = int(1.0 / PIXEL_HA)
```

```
lbl, n = ndimage.label(pred_f1)
```

```
sizes = ndimage.sum(np.ones_like(lbl), lbl, range(1, n + 1))
```

```
keep = {i + 1 for i, s in enumerate(sizes) if s >= min_px}
```

```
pred_filt = np.isin(lbl, list(keep)) if keep else np.zeros_like(pred_f1)
```

```
# [3] 필터를 거친 마스크를 같은 함수로 다시 채점함
```

```
iou1, p1, r1, d1 = iou_prf(pred_filt, true_change)
```

```
# 출력: [거짓변화 필터] NDVI 차분>0.25 + 최소면적≥1ha(100px)
```

```
#           필터 후: IoU 0.865 | 정밀도 1.000 | 재현율 0.865 | Dice 0.927
```

```
#           → 정밀도 0.572 → 1.000 (개선 +0.428), IoU 0.525 → 0.865
```

- 코드가 하는 일을 줄 단위로 보면

코드	하는 일
<code>np.abs(ndvi_t2 - ndvi_t1)</code>	두 시점의 식생지수를 칸마다 빼고 부호를 떼어 변화의 크기 만 남김. 늘었든 줄었든 달라진 폭을 봄
<code>dndvi > 0.25</code>	그 폭이 0.25를 넘는 칸만 참으로 하는 격자를 만들
<code>np.logical_and(pred, ndvi_ok)</code>	예측도 1이고 식생 변화도 큰 칸만 남김. 둘 중 하나라도 아니면 지워짐
<code>int(1.0 / PIXEL_HA)</code>	1헥타르가 몇 픽셀인지 계산. 픽셀 하나가 0.01헥타르이므로 $1 \div 0.01 = 100$ 픽셀
<code>ndimage.label</code>	맞닿은 1 픽셀 덩어리마다 1, 2, 3... 번호를 매기고 번호 격자와 덩어리 개수를 돌려줌(7.1)
<code>ndimage.sum(...)</code>	번호별로 칸 수를 세어 각 덩어리의 크기를 구함
<code>keep</code>	크기가 100픽셀 이상인 덩어리의 번호만 모은 집합
<code>np.isin(lbl, list(keep))</code>	번호 격자에서 남길 번호의 칸만 참으로 하는 최종 마스크를 만들

- 위 출력은 `results/6-1-change-detection-policy.log` 에 저장된 실행 기록으로, 지표 해설 문구를 고치기 전에 돌린 것임. 지금 코드는 필터 전후 재현율을 함께 찍어 해설 줄의 표현이 로그와 다르고, IoU·정밀도·재현율·Dice 수치는 같음
- 화면에 나온 값을 혼동행렬 관점으로 읽을 것
- 필터 전후 혼동행렬 (실제 변화 픽셀 1,000개 기준으로 역산한 값). 비율을 읽기 쉽게 1,000개로 맞추는 것이지 원자료의 2,001개를 버린 것이 아님

구분	TP (잡았다)	FP (헛짚었다)	FN (놓쳤다)	정밀도	재현율	IoU
필터 전	865	647	135	0.572	0.865	0.525
필터 후	865	0	135	1.000	0.865	0.865
변화량	0	-647	0	+0.428	0.000	+0.340

- 표의 세 숫자를 식에 넣으면 지표 값이 그대로 나옴
 - 필터 전 정밀도 = $865 \div (865 + 647) = 865 \div 1,512 = \mathbf{0.572}$
 - 필터 전 재현율 = $865 \div (865 + 135) = 865 \div 1,000 = \mathbf{0.865}$
 - 필터 전 IoU = $865 \div (865 + 647 + 135) = 865 \div 1,647 = \mathbf{0.525}$
 - 필터 후 정밀도 = $865 \div (865 + 0) = \mathbf{1.000}$ — 분모에서 FP만 빠졌음
 - 필터 후 재현율 = $865 \div (865 + 135) = \mathbf{0.865}$ — 분자도 분모도 안 바뀌었음
 - 필터 후 IoU = $865 \div (865 + 0 + 135) = 865 \div 1,000 = \mathbf{0.865}$
 - 필터는 **FP(헛짚은 픽셀) 647개를 제거**했을 뿐임. TP와 FN은 그대로이므로, 정밀도($TP/(TP+FP)$)만 오르고 재현율($TP/(TP+FN)$)은 변하지 않음

- 이유는 계산 구조에 있음. 재현율의 식에는 FP가 아예 들어가지 않으므로, FP를 몇 개 지우든 값이 움직이지 않음
- 더 근본적으로는 모델이 애초에 놓친 픽셀(FN=135)이 예측 안에 존재하지 않음. **예측을 깎는 어떤 연산(FP 제거)으로도 FN을 되살릴 수 없음**
- 재현율을 올리려면 판정 임계를 낮추거나 학습 설정을 바꿔야 함. 후처리 단계에서는 방법이 없음
- 필터가 TP를 건드리지 않은 이유 (FP만 걸러진 이유)
 - 사전 생성 데이터의 개발 패치(TP)는 NDVI가 0.65대 → 0.15대로 급락해 Δ NDVI가 임계 0.25 를 크게 넘고, 반지름 9~14픽셀 원형이라 넓이도 최소면적 100px 을 여유 있게 넘김 → **두 필터 조건을 모두 통과**
 - 반면 거짓양성(FP)은 계절성 잡음 위에 뿌린 산발 픽셀이라 Δ NDVI가 임계에 못 미쳐 첫 조건에서 걸러짐 → **647개 전부 제거**
 - 재현율이 1.000 에 못 미치는 원인(FN=135)은 필터가 아니라 예측 마스크를 만들 때 준 경계 침식임. 이 손실은 필터를 걸기 전부터 있었음
 - **다른 자료에서는 필터가 TP 경계까지 깎아 FN이 늘고 재현율이 떨어질 수 있음.** 코드가 필터 전후 재현율을 나란히 찍는 이유가 그것을 눈으로 확인하라는 것임

6.4 정확도 하나만 보면 안 되는 이유

- 변화 픽셀이 전체의 1% 안팎인 자료를 생각함. **모든 픽셀을 배경이라고만 답하는 예측기의 정확도는 99%임**
 - 정확도는 $(TP + TN) \div$ 전체 칸 수임. 전부 배경이라고 답하면 TP는 0이지만 TN이 99%를 채우므로 값이 0.99가 됨
 - 그런데 이 예측기의 재현율은 0임. 변화를 하나도 못 잡았는데 정확도만 보면 우수한 모델로 보임
- 5장에서 Macro-F1을 함께 본 이유와 같은 구조임. 클래스가 크게 불균형한 자료에서는 정확도가 다수 클래스의 비율을 그대로 보고할 뿐이므로, **IoU·정밀도·재현율처럼 관심 클래스만 놓고 세는 지표**를 씀
 - 학습 단계에서도 같은 문제가 나타나므로 세그멘테이션에서는 불균형을 다루는 전용 손실함수를 쓰지만, 그 설계는 이 장의 범위 밖임

7. 마스크를 객체로 바꾸기

- 채점을 통과한 마스크는 아직 픽셀의 집합임. 산출 단위를 객체로 정한 과제에서는 이 집합을 객체 목록으로 바꾸고 객체마다 면적·형상·접도를 재야 결정에 씀
- 여기서 **필지**는 등록된 땅 한 덩어리를, **접도**는 그 땅의 경계 가운데 도로에 닿는 길이를, **맹지**는 도로에 닿는 경계가 없어 들어갈 길이 없는 땅을 뜻함
- 두 번째 코드가 정답을 포함한 도시 블록 래스터를 읽어 객체화·기하 피쳐·요건 필터·편향 보정·대조군·절단선을 한 번에 계산함. **7절부터 10절까지의 표와 출력은 모두 이 한 번의 실행에서 나옴**

python lecture_practice/chapter6/code/6-2-site-sourcing.py

```
# 출처: lecture_practice/chapter6/results/6-2-site-sourcing.log [1]
```

무대: 512×512 래스터(픽셀 1m=1㎡, 26.2ha) | 도로 50,005㎡ | 건물 26,877㎡

개발 요건(예시 설정): 면적 ≥ 1,000㎡, 접도 ≥ 12m, 최소폭 ≥ 15m

[1] 정답: 유휴 부지 32필지 중 요건을 모두 채우는 우량 부지 10필지

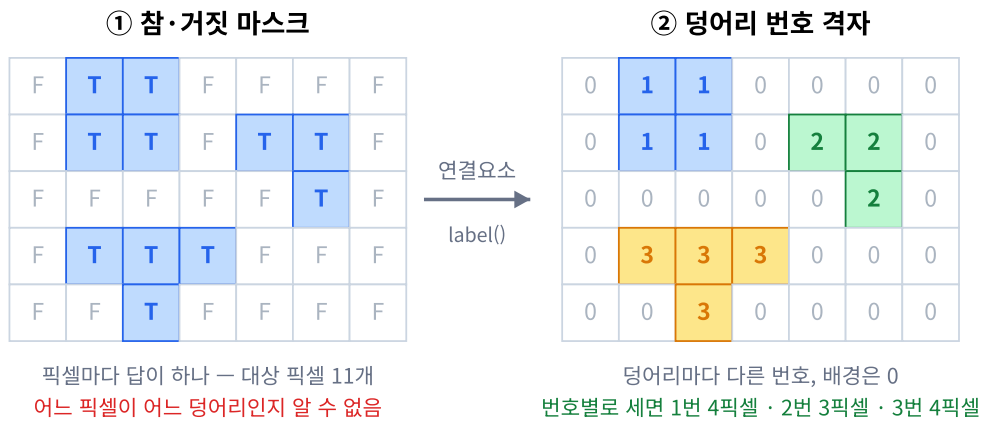
탈락 사유 - 면적 미달 9 | 맹지·접도 부족 10 | 최소폭 미달 4 (중복 포함)

- 픽셀 한 변이 1m이므로 픽셀 하나가 1㎡이고(6절의 10m 격자와 달리 픽셀 수가 곧 제곱미터임), $512 \times 512 = 262,144\text{㎡}$ 이므로 약 26.2헥타르임
- 탈락 사유의 합($9 + 10 + 4 = 23$)이 탈락 필지 수($32 - 10 = 22$)보다 큰 것은 중복 때문임. 한 필지가 면적도 부족하고 접도도 부족할 수 있음
- `urban_block.npz`에는 정답 필지와 예측 마스크가 함께 들어 있으므로 채점이 가능함. 자료에는 경계 침식, 오탐, 과분할을 넣어 두었고, 8.3의 대조군은 이 셋을 하나씩만 빼서 돌린 결과임

7.1 연결요소 라벨링

- 서로 맞닿은 픽셀을 하나의 덩어리로 묶고 번호를 매기는 계산이 **연결요소**(connected component)임
 - 결과는 번호가 적힌 격자임. 대상인 칸은 전부 어느 덩어리에 속하는지가 1, 2, 3... 번호로 바뀌고, 배경인 칸은 0으로 남음
 - 무엇을 "맞닿았다"고 볼지 두 정의가 있음
 - **4-이웃**은 상하좌우만 이웃으로 셈
 - **8-이웃**은 대각선도 이웃으로 셈
 - 모서리 하나로만 이어진 두 덩어리는 8-이웃에서 한 객체, 4-이웃에서 두 객체가 됨

같은 격자에 참·거짓을 적을 때와 덩어리 번호를 적을 때



③ 무엇을 "맞닿았다"고 볼지에 따라 번호가 갈린다



그림 6.5 같은 격자에 참·거짓을 적을 때와 덩어리 번호를 적을 때의 차이. 왼쪽은 대상 픽셀 11개를 모두 참으로만 적어 덩어리를 구별하지 못하고, 오른쪽은 덩어리마다 1·2·3을 적어 번호별로 픽셀을 세면 덩어리별 면적이 나옴. 아래는 모서리만 닿은 두 덩어리로, 4-이웃에서는 번호가 둘, 8-이웃에서는 하나가 됨. 설명용 예시 격자임

- 가늘고 긴 대상이 계단 모양으로 이어지는 도로·하천에서는 8-이웃이 끊김을 줄이고, 객체가 조밀한 도심 건물에서는 4-이웃이 병합을 줄임. 대각선으로 닿은 두 건물을 한 채로 세지 않으려는 것임

7.2 최소 크기 필터

- 라벨링 결과에는 잡음이 만든 작은 덩어리가 다수 섞임. 이를 미리 빼지 않으면 이후 통계가 얼룩의 개수에 지배됨
 - 평균 면적·객체 수·분포가 전부 얼룩 쪽으로 끌려감. 아래 실행에서 얼룩이 909개, 실제 대상이 40개임

```
# 주석에 붙인 출력의 출처: lecture_practice/chapter6/results/6-2-site-sourcing.log
from scipy import ndimage

# [1] 예측 마스크를 연결요소로 나눔
lab, _ = ndimage.label(pred)

# [2] 50m² 미만 얼룩은 후보가 될 수 없으므로 객체 표에서 미리 뺌
tab = object_table(lab, dist_road, min_area=50)
n_speckle = int(lab.max()) - len(tab)

# 출력: [2] 예측 마스크 객체화: 연결요소 949개 중 909개는 50m² 미만 얼룩(제외) → 분석 대상 40개
```

- `lab.max()` 는 마지막으로 매긴 번호이므로 덩어리의 총 개수(949)임. `object_table(...)` 은 번호별로 면적 등을 재서 표로 만들되 `min_area=50` 미만은 행을 만들지 않으므로 남은 행이 40개이고, $949 - 40 = 909$ 가 걸러진 얼룩 수임
- 화면에 나온 값에서 읽을 것
 - 연결요소가 **949개** 나왔는데 그중 **909개가 50m² 미만 얼룩**이었고 분석 대상은 **40개**만 남음
 - 객체 수의 96%($909 \div 949 = 0.958$)가 후처리로 걸러지는 잡음이라는 뜻이며, **연결요소의 개수 자체가 대상의 개수를 뜻하지 않음**

7.3 기하 피쳐 다섯 가지

- 객체가 되면 다섯 값을 잴. 결정에 쓰는 값이 면적만이 아니기 때문임

피쳐	계산	결정에서의 쓰임
면적 A	픽셀 수 × 픽셀 면적	최소 면적 요건 판정
둘레 P	크랙 둘레(픽셀 사이 경계선 길이)	조밀도 계산, 면적 편향 보정
조밀도	$4\pi A/P^2$ (원=1, 정사각형 ≈ 0.785)	형상 선별의 참고 지표
최소폭	객체에 들어가는 최대 내접원의 지름	최소폭 요건 판정
접도 길이	도로에서 허용오차 안에 있는 경계 픽셀 수	접도 요건 판정

- 다섯 값을 객체마다 계산하는 부분이 `object_table` 임
- 다섯 중 셋은 이름만으로는 무엇을 센 값인지 알 수 없으므로 따로 적음
- **크랙 둘레**는 경계에 놓인 픽셀의 개수가 아니라, 1인 칸과 0인 칸 사이에 생기는 **경계선 토막의 길이를 모두 더한 값**임
 - 40m × 25m 직사각형이면 위·아래 변이 각 40m, 좌·우 변이 각 25m이므로 $2 \times (40 + 25) = 130\text{m}$ 임

- 경계 픽셀을 세는 방식과 값이 다름. 모퉁이 칸은 두 변이 노출되므로 토막이 둘이지만 칸으로는 하나임
- 둘레를 이렇게 세는 이유는 9.3의 보정에서 "테두리 1m당 몇 제곱미터가 깎였나"를 추정하는 데 쓰기 때문임. 길이 단위로 재야 면적과 곱셈 관계가 맞음
- **조밀도**는 $4\pi A/P^2$ 이고, 같은 둘레로 가장 넓은 모양이 원이라는 성질을 써서 원이 1이 되게 맞춘 비율임
 - 원이면 $A = \pi r^2$, $P = 2\pi r$ 이므로 $4\pi \times \pi r^2 \div (2\pi r)^2 = 4\pi^2 r^2 \div 4\pi^2 r^2 = 1$
 - 정사각형이면 $A = s^2$, $P = 4s$ 이므로 $4\pi \times s^2 \div (4s)^2 = 4\pi s^2 \div 16s^2 = \pi \div 4 \approx 0.785$
 - 같은 면적을 길게 늘이면 둘레만 커지고 분모가 제곱으로 커지므로 값이 0에 가까워짐. 그래서 "뭉쳐 있는가, 길쭉한가"를 한 숫자로 읽을 수 있음
- **최소폭**은 객체 안에 들어가는 가장 큰 원의 지름으로 대신 낸 값이고, 그 원은 **거리변환**으로 찾음
 - 거리변환은 1인 칸마다 "가장 가까운 0인 칸까지 몇 칸인가"를 적어 넣는 계산임. 가장자리 칸은 1에 가깝고 가운데로 갈수록 값이 커짐
 - 그 값이 가장 큰 칸이 객체에서 가장 깊은 곳이고, 그 값이 거기서 그릴 수 있는 가장 큰 원의 반지름임. 지름은 그 값의 두 배임
 - 격자는 칸 단위로 끊겨 있어 이 지름이 실제보다 약 1m 크게 나옴. 코드 주석이 "격자 이산화로 약 1m 과대"라고 적어 둔 것이 그것임

```
# 코드 출처: lecture_practice/chapter6/code/6-2-site-sourcing.py
edt = ndimage.distance_transform_edt(sub)
boundary = sub & ~ndimage.binary_erosion(sub)
rows.append({
    "obj_id": i,
    "area_m2": area,
    "perim_m": float(perim),
    # 조밀도 4πA/P² : 원=1, 정사각형≈0.785, 길쭉할수록 0에 가깝다
    "compactness": float(4 * np.pi * area / perim ** 2) if perim else np.nan,
    "aspect": float(max(h, w) / min(h, w)),
    # 최소폭 대리값: 내접원 지름(거리변환 최댓값×2). 격자 이산화로 약 1m 과대.
    "min_width_m": float(2 * edt.max()),
    "frontage_m": float((dist_road[big][boundary] <= tol).sum()),
    # 허용오차 없이 '도로에 직접 맞붙은' 경계만 세면 얼마가 되는가
    "frontage_strict_m": float((dist_road[big][boundary] <= 1.0).sum()),
})
```

□ 코드가 하는 일을 줄 단위로 보면

코드	하는 일
<code>distance_transform_edt(sub)</code>	위에서 말한 거리변환. <code>sub</code> 는 이 객체만 1로 둔 작은 격자
<code>sub & ~binary_erosion(sub)</code>	<code>binary_erosion</code> 이 테두리를 한 겹 깎은 격자를 만들고, 원래 격자에서 그것을 빼면 테두리 한 겹만 남음 → 경계 칸 <code>boundary</code>
<code>2 * edt.max()</code>	<code>edt.max()</code> 가 가장 깊은 곳까지의 거리이므로 그 두 배가 내접원 지름, 즉 최소폭
<code>dist_road[big][boundary]</code>	<code>dist_road</code> 는 칸마다 가장 가까운 도로까지의 거리를 담은 격자. 이 객체의 경계 칸에서 그 거리를 뽑음
<code>(... <= tol).sum()</code>	거리가 허용오차 안인 경계 칸을 셈. 픽셀 한 변이 1m이므로 개수가 곧 접도 길이(m)
<code>frontage_strict_m</code>	같은 계산을 허용오차 없이(<code><= 1.0</code>) 한 값. 7.4에서 두 값을 비교함
<code>aspect</code>	객체를 감싸는 사각형의 긴 변 ÷ 짧은 변. 이 장의 요건 판정에는 쓰지 않음

○ 계산 예 1 — 40m×25m 직사각형 필지: 면적 1,000m², 크랙 둘레 2×(40+25)=130m, 조밀도

$4\pi \times 1,000 / 130^2 \approx 0.743$, 최소폭은 짧은 변인 25m

○ 계산 예 2 — 100m×10m 필지: 면적은 같은 1,000m²인데 둘레 220m, 조밀도 0.260, 최소폭 10m

○ 두 필지는 면적이 같지만 최소폭 15m 요건에서 뒤쪽만 탈락함. 면적 하나로는 가늘고 긴 자투리땅을 걸러 낼 수 없음. 길이가 100m라도 폭이 10m면 건물을 앉힐 수 없기 때문임

□ 그런데 요건으로 거르기 전에, 가는 부분은 마스크에 담기지도 않음

○ 몸통 12m×8m에 길이 12m 복도를 붙인 건물을 1m 격자로 래스터화하고, 경계선이 가로지른 칸을 배경으로 판정했을 때 복도가 몇 칸 남는지 셈. 격자가 놓인 위치 25가지(0.2m 간격)를 훑은 값임

복도 폭	참 면적	보통 래스터	경계 칸을 배경으로	통째로 사라진 격자 위치
3.0m (3픽셀)	36m ²	36.2칸	22.0칸	0/25
2.0m (2픽셀)	24m ²	22.6칸	13.2칸	0/25
1.5m (1.5픽셀)	18m ²	18.0칸	4.4칸	15/25
1.0m (1픽셀)	12m ²	13.4칸	0.0칸	25/25
0.8m (0.8픽셀)	9.6m ²	8.9칸	0.0칸	25/25

○ 폭이 2픽셀 이상이면 복도가 남음. 1.5픽셀로 내려가면 격자가 놓인 위치에 따라 25번 중 15번 통째로 사라지고, 1픽셀 아래에서는 25번 모두 사라짐

○ 이유는 간단함 — 폭이 2픽셀 미만이면 복도 안에 통째로 안쪽인 칸이 하나도 없어 모든 칸이 경계 칸이 됨. 경계 칸을 깎으면 남는 것이 없음

- 그래서 최소폭은 요건 판정에만 쓰는 값이 아님. 그 값이 픽셀 크기의 2배를 밑돌면 그 부분은 애초에 켤 수가 없음. 4.1에서 본 1.2픽셀 주택이 같은 이야기임

7.4 허용오차는 침식 깊이보다 크게 잡는다

- 도로에 직접 맞닿은 경계만 접면으로 세면 계산은 정확해 보이지만 결과가 뒤집힘
- 이유를 단계로 나누면
 1. 경계 침식이 객체 가장자리를 한 겹 깎음
 2. 도로에 붙어 있던 **진짜 객체**가 도로에서 물러난 것으로 보임
 3. 침식되지 않은 **오탐 덩어리**는 원래 자리에 그대로 있어 도로에 붙은 채 남음
 4. 결과적으로 진짜만 탈락하고 가짜만 살아남음
- 3단계가 성립하는 이유: 침식은 정답 필지의 마스크에만 걸어 두었으므로 오탐 덩어리는 깎이지 않았음
- 같은 실행이 두 판정을 나란히 찍음 — 허용오차를 둔 `frontage_m` 과 두지 않은 `frontage_strict_m` 임

출처: lecture_practice/chapter6/results/6-2-site-sourcing.log [3]

[3] 점도 판정 허용오차: 도로에 직접 맞닿은 경계만 세면 분석 대상 40개 중 37개가 접면 부족으로 탈락한다
남는 3개 중 진짜 필지는 0개 - 침식이 진짜 부지를 도로에서 한 겹 떼어 놓는 사이, 침식되지 않은 오탐만
도로에 붙어 살아남는다.
→ 허용오차는 침식 깊이보다 커야 한다. 여기서는 3m를 쓴다.

- 화면에 나온 값에서 읽을 것
 - 허용오차를 0으로 두면 분석 대상 40개 중 **37개**가 접면 부족으로 탈락하고, 남은 **3개 중 진짜 필지는 0개**임. 위 네 단계가 그대로 숫자로 나타난 결과임
 - 그래서 도로에서 일정 거리 안을 접면으로 인정하는 허용오차를 두고, 그 크기를 마스크의 침식 깊이보다 크게 잡음
 - 숫자로 보면 침식이 한 겹일 때 진짜 필지는 도로에서 약 1m 물러나 있으므로 허용오차가 1m보다 커야 그 접면을 되살림. 자료에는 두 겹 침식(약 2m) 필지도 있으므로 그보다도 커야 함
 - 이 실습에서 쓴 허용오차는 `FRONT_TOL = 3.0`, 즉 3m임. 두 겹 침식까지 덮는 값임
 - 이것은 임계를 느슨하게 하는 타협이 아니라, 재는 자리가 한 겹 밀려 있다는 사실을 판정식에 반영하는 절차임

8. 객체 단위로 채점하기

- 픽셀이 얼마나 겹치는지만 보면 "필지를 몇 개 찾았는가"와 "각 필지의 면적이 맞는가"를 알 수 없음.
객체로 나눈 뒤에는 객체끼리 짝을 지어 채점함

8.1 매칭 규칙과 네 가지 지표

- 객체 단위 채점의 첫 작업은 예측 객체와 정답 객체를 대응시키는 **매칭 규칙**을 정하는 것임. 이 장에서는 어떤 예측 객체가 정답 객체 면적의 절반 이상을 덮으면 그 둘을 대응으로 봄
 - 매칭이 끝나면 네 가지를 셈

지표	낮거나 클 때의 뜻
검출률	낮으면 객체 자체를 놓쳤다는 뜻
오탐 객체 수	크면 목록에 없는 대상이 섞임
과분할·병합	크면 객체 수와 규모가 함께 틀림
면적 상대오차	부호가 한쪽으로 쏠리면 체계적 편향

- 한 정답 객체와 겹치는 예측이 둘 이상이고 어느 것도 정답 면적의 절반을 못 넘기면 **과분할**임. 한 필지가 여러 조각으로 쪼개진 경우임
- 반대로 예측 하나가 서로 다른 정답 둘 이상을 각각 절반 넘게 덮으면 **병합**임. 옆 필지까지 한 덩어리로 붙인 경우임
- 코드에서 이 두 임계가 상수로 적혀 있고, 둘은 **무엇을 분모로 보느냐**가 다름
 - `MATCH_OBJ = 0.20` 은 **객체 쪽에서** 보는 기준임. 예측 객체 면적의 20% 이상이 어느 필지와 겹쳐야 그 필지의 객체로 봄. 도로를 조금 물고 있는 객체도 필지에 귀속시키려는 느슨한 기준임
 - `MATCH_PARCEL = 0.50` 은 **필지 쪽에서** 보는 기준임. 필지 면적의 절반 이상이 덮여야 그 필지를 찾았다고 봄. 조각 하나로 필지를 찾았다고 세지 않으려는 엄격한 기준임
 - 두 기준의 방향이 다르기 때문에 "객체는 이 필지에 귀속되었지만 필지는 검출되지 않았다"가 성립함. 과분할이 그 상태임
- 7절에서 실행한 코드가 네 지표를 함께 찍음

```
# 출처: lecture_practice/chapter6/results/6-2-site-sourcing.log [2]
```

```
필지 검출률 30/32 (0.938) | 정답 필지와 무관한 오탐 객체 5개
```

```
과분할(한 필지가 여러 객체로 쪼개짐) 3필지 | 병합(한 객체가 두 필지 이상을 덮음) 0객체
```

- 화면에 나온 값에서 읽을 것
 - 검출률 `0.938` 은 32필지 중 30필지를 찾았다는 뜻이고($30 \div 32 = 0.938$), 여기에 정답과 무관한 오탐 객체가 `5개` 더 얹혀 있음
 - 과분할이 `3필지`, 병합이 `0객체` 임. 과분할된 필지는 조각마다 면적이 따로 잡히므로 규모가 절반 안팎으로 축소되어 보고됨

8.2 오차의 방향: 침식은 평균해도 남는다

- 면적 상대오차는 중앙값과 사분위수를 함께 봄. 평균만 보면 큰 오탐 하나가 값을 끌고 감
 - **면적 상대오차**는 (추정 면적 - 참 면적) ÷ 참 면적임. 음수면 작게 잤다는 뜻이고, -0.125면 12.5% 작게 잤다는 뜻임
 - **중앙값**은 값을 크기순으로 줄 세웠을 때 가운데 값이고, **사분위수**는 그 줄을 넷으로 나눈 경계값임. 1사분위는 아래에서 25% 지점, 3사분위는 75% 지점임

출처: lecture_practice/chapter6/results/6-2-site-sourcing.log [2]

면적 상대오차(검출 30필지): 중앙값 -0.125 (1사분위 -0.177 / 3사분위 -0.112) - 침식이 면적을 체계적으로 깎는다

오차는 크기에 따라 다르다 - 큰 절반 -0.112 / 작은 절반 -0.136 (둘레÷면적 비가 작은 필지에서 손실이 작다)

과분할 필지 1: 참 2,511㎡ → 조각 2개(최대 1,026㎡, 합 2,026㎡)

과분할 필지 2: 참 2,412㎡ → 조각 2개(최대 1,093㎡, 합 2,152㎡)

과분할 필지 9: 참 2,394㎡ → 조각 2개(최대 1,216㎡, 합 2,105㎡)

- 화면에 나온 값에서 읽을 것
 - 중앙값 -0.125 에 1사분위 -0.177, 3사분위 -0.112 임. **사분위수까지 전부 음수라는 점이 이 출력의 핵심임** — 부호의 쓸림이 무작위 오차와 체계적 편향을 가르는 기준임
 - 3사분위가 -0.112라는 것은 **가장 잘 켜 쪽 25%조차 11.2% 이상 작게 잤다**는 뜻임. 크게 켜 필지가 하나도 없음
 - 무작위 오차라면 크게 켜 쪽과 작게 켜 쪽이 섞여 표본을 늘리면 평균이 0으로 모임. 경계 침식처럼 항상 면적을 깎는 오차는 평균해도 남음
 - 큰 절반 -0.112, 작은 절반 -0.136 으로 편향의 크기가 객체 크기에 달려 있음. 4.1에서 본 대로 경계 한 겹이 지우는 픽셀 수는 대략 둘레만큼이므로 정사각형 대상의 상대 손실은 4/한 변의 길이이고, **같은 침식 깊이라도 작은 객체가 상대적으로 더 많이 깎임**
 - 과분할된 세 필지는 참 면적 2,394~2,511㎡인데 가장 큰 조각만 보면 1,026~1,216㎡로, 참 면적 대비 0.41~0.51임. 조각 하나만 보고하면 필지 크기를 절반 안팎으로 잘못 세계 됨
 - 조각을 합쳐도 2,026~2,152㎡로 참값에 못 미침. 갈라 놓은 폭 2m 띠와 가장자리 침식이 함께 면적을 가져갔기 때문임

주의. 총량이 맞는 것과 객체별로 맞는 것은 다름. 7절 첫머리의 출력에서 예측 마스크의 총 면적이 정답의 1.170배였는데, 여기서는 개별 필지의 면적이 중앙값 12.5% 작게 나옴. 오탐이 총량을 부풀리고 침식이 개별 객체를 깎으면 두 오차가 상쇄되어 총량 비교로는 아무것도 드러나지 않음.

8.3 원인을 분리하는 대조군

- 성능표 한 줄로는 잔여 오차가 어디서 왔는지 알 수 없음. 마스크에는 경계 침식, 오타, 과분할이 동시에 들어 있기 때문임
 - 원인을 분리하려면 주장하려는 메커니즘 하나만 제거한 조건을 만들어 같은 파이프라인을 다시 돌림. 다른 조건은 그대로 두고 하나만 빼는 것이 핵심임

조건	제거하는 오차	확인하는 주장
본 실험	없음	기준
C1	경계 침식	면적 편향이 침식에서 왔는가
C2	오타	목록에 섞인 헛후보가 오타에서 왔는가
C3	과분할	객체 수·규모 오차가 과분할에서 왔는가

- 대조군 없이 "침식 때문에 후보를 놓쳤다"고 쓰면 그것은 해석이지 측정이 아님. C1에서 오타락이 0이 되어야 그 주장이 성립함
- 이 자료에서는 세 오차를 직접 심었으므로 하나만 빼는 조건을 만들 수 있음. 실제 자료에서는 오차를 뺄 수 없으므로, 같은 역할을 하려면 일부 대상을 실측해 비교군으로 삼음
- 10절이 이 세 대조군의 실측을 담고 있음

9. 구역별로 모아 결정으로 바꾸기

9.1 집행 단위 집계

- 픽셀 마스크는 그 자체로 행정·사업의 결정 단위가 아님. "어느 픽셀 몇 개가 변했다"로는 담당 부서도 예산 항목도 정해지지 않음
 - 세 가지를 순서대로 정함 — 단위 경계(행정구역·보호구역·필지 등), 마스크와 경계를 포갠 단위별 픽셀 수, 픽셀 수를 면적으로 환산
- 6절에서 실행한 `6-1-change-detection-policy.py` 가 이어서 이 계산을 함. 128×128 래스터를 $4 \times 4 = 16$ 개 구역으로 나누고 필터를 거친 마스크를 포갠

```
# 주석에 붙인 출력의 출처: lecture_practice/chapter6/results/6-1-change-detection-policy.log
# [1] 단위 경계 - 행과 열을 4등분해 구역 번호를 매김
zone = (np.arange(H)[: , None] // (H // N_ADMIN)) * N_ADMIN + (np.arange(W)[None, : ] // (W // N_ADMIN))

# [2] 구역별 픽셀 수를 세고 [3] 픽셀 면적을 곱해 헥타르로 환산
for z in range(N_ADMIN * N_ADMIN):
    zmask = zone == z
    changed_px = int(np.logical_and(pred_filt, zmask).sum())
    rows.append({"admin_zone": z, "changed_px": changed_px,
                "changed_ha": round(changed_px * PIXEL_HA, 2)})
```

□ 첫 줄이 이 장에서 가장 뻑뻑한 코드이므로 조각으로 끊어 읽음. `H` 와 `W` 는 128이고 `N_ADMIN` 은 4이므로 `H // N_ADMIN` 은 32임

조각	하는 일	결과
<code>np.arange(H)</code>	행 번호 목록을 만듦	0, 1, ..., 127
<code>// 32</code>	32로 나눈 몫만 남김. 행 0~31은 0, 32~63은 1, 64~95는 2, 96~127은 3	행 구역 번호 0~3
<code>[:, None]</code>	그 목록을 세로 한 줄로 세움	128행 × 1열
<code>* N_ADMIN (× 4)</code>	행 구역 번호에 4를 곱해, 한 행 구역 안에 열 구역 4개가 들어갈 자리를 비움	0, 4, 8, 12
<code>np.arange(W)[None, :]</code> <code>// 32</code>	같은 계산을 열에 해서 가로 한 줄로 만듦	열 구역 번호 0~3, 1행 × 128열
두 줄을 +	NumPy가 둘을 128 × 128로 펼쳐 모든 조합을 한 번에 계산	칸마다 (행 구역 × 4 + 열 구역) = 0~15

- 예를 들어 행 70, 열 100인 칸은 행 구역 2, 열 구역 3이므로 $2 \times 4 + 3 = 11$ 번 구역임
- 둘째 조각은 단순함. `zone == z` 가 `z`번 구역인 칸만 참으로 하고, `np.logical_and(pred_filt, zmask)` 가 그 구역에서 변화로 남은 칸만 참으로 함. 그 수에 픽셀 면적 0.01헥타르를 곱해 헥타르로 바꿈

출처: lecture_practice/chapter6/results/6-1-change-detection-policy.log [6.6 정책]

admin_zone	changed_px	changed_ha	priority_rank
9	293	2.93	1
10	247	2.47	2
13	230	2.30	3
6	178	1.78	4
15	141	1.41	5
0	134	1.34	6
7	124	1.24	7
1	92	0.92	8
4	92	0.92	9
5	70	0.70	10
8	35	0.35	11
14	32	0.32	12
11	31	0.31	13
12	30	0.30	14
2	1	0.01	15

탐지된 개발 변화 총 17.3ha, 우선순위 저장 → change_admin_priority.csv

□ 화면에 나온 값에서 읽을 것

- `changed_ha` 는 `changed_px` 에 0.01을 곱한 값임. 구역 9는 293픽셀 × 0.01 = 2.93헥타르임
- 16개 구역 중 15개에 변화가 잡히고 합계가 17.3헥타르 임. 표에 찍히지 않은 구역 3은 변화 픽셀이 0이라 `changed_ha > 0` 조건에서 빠졌고, 저장된 CSV에는 16행이 그대로 들어 있음
- 상위 세 구역이 2.93 · 2.47 · 2.30 헥타르로 촘촘함. 구역 8위와 9위는 둘 다 0.92 헥타르로 같은 값이라 순위가 정렬 순서에 따라 갈림 — **순위표를 읽을 때는 값의 간격을 함께 봄**. 1픽셀 차이로 뒤집히는 순위는 근거로 쓸 수 없음
- **단위 경계를 어떻게 잡느냐가 순위를 바꿈**. 여기서는 격자로 나눴지만 실제 행정경계로 나누면 상위 구역이 달라질 수 있음. 큰 변화 하나가 경계에 걸치면 두 구역으로 쪼개져 둘 다 순위가 내려감

9.2 정렬형과 절단형은 오차에 다르게 반응한다

□ 결정의 형태가 둘임 — 위에서부터 자원을 배분하는 **정렬형**, 요건을 만족하는 것만 남기는 **절단형**

- 정렬형에서 면적 오차는 순위를 흔들. 3순위 구역이 5순위로 내려가도 그 구역은 표에 남아 있고 다음 회차에 다시 검토됨
- 절단형에서는 임계 근처의 대상이 목록에서 **아예 사라짐**. 사라진 대상은 다시 확인되지 않으며, **오탈락이 발생했다는 사실 자체가 산출물에 남지 않음**

- **오탈락**은 요건을 실제로는 만족하는데 추정값이 임계 아래로 내려가 목록에서 빠진 것을 말함. 참 면적이 1,100m²인 필지가 12.5% 깎여 $1,100 \times 0.875 = 963\text{m}^2$ 로 추정되면 1,000m² 선에서 떨어지고, 결과표에는 그 필지의 이름조차 남지 않음
- 반대 방향의 오류는 **헛후보**임. 요건을 만족하지 않는데 목록에 올라온 것이고, 이쪽은 현장에 나가 보면 드러남. 오탈락은 나가 보지 않으므로 드러나지 않는다는 점이 다름
- 이 차이 때문에 절단형 결정에는 두 계산이 더 필요함 — 체계적 편향을 되돌리는 보정(9.3), 임계 자체를 비용으로 계산하는 절차(9.4)
 - 어느 형태든 탈락하거나 순위에서 밀린 대상의 목록을 별도 파일로 남김. 이 목록이 없으면 오탈락을 사후에 셀 수 없음. 10절의 실습은 `ch6_site_candidates.csv` 와 `ch6_missed_sites.csv` 두 파일로 이를 남김

9.3 체계적 편향의 보정

- 면적 상대오차의 부호가 쏠린 것을 확인했다면 소수의 대상을 실측해 그 크기를 되돌림. 전체를 다시 재는 대신 표본으로 계수를 구하는 것임. 두 형태를 씀
 - **스칼라 보정**: 표본에서 참 면적을 추정 면적으로 나눈 값의 평균 k 를 모든 대상의 추정 면적에 곱함
 - **둘레 모형**: 침식이 없애는 픽셀 수가 둘레에 비례한다는 성질(4.1)을 써서 보정 면적을 $A + \hat{c}P$ 로 두고 $\hat{c}$ 를 표본에서 추정함. 한 겹 침식이면 이론값 $\hat{c} \approx 1$ 임
 - 둘레 모형은 둘레가 긴 대상에 더 많이 더해 주므로, 같은 면적이라도 길쭉한 땅과 네모난 땅을 다르게 되돌림
 - 두 보정 모두 **전역 계수 하나**를 쓰므로 대상마다 다른 침식 깊이를 완전히 되돌리지 못함

```
# 주석에 붙인 출력의 출처: lecture_practice/chapter6/results/6-2-site-sourcing.log
# [1] 스칼라 보정 - 표본에서 참/추정 비의 평균
k_cand = float((sdf["true"] / sdf["est"]).mean())

# [2] 둘레 모형 - 침식이 없애는 픽셀 수가 둘레에 비례한다고 보고 c-hat를 최소제곱으로 추정
c_hat = float((sdf["perim"] * (sdf["true"] - sdf["est"])).sum()
              / (sdf["perim"] ** 2).sum())

# 출력: ① 스칼라 보정 k = 평균(참/추정) = 1.128
#       ② 둘레 보정 A-hat = A + c-hat * P, c-hat = 0.916 (침식 한 겹이면 c-hat ≈ 1이 이론값)
```

- 코드가 하는 일
 - `sdf` 는 실측한 표본만 담은 표이고, `true` 는 실측한 참 면적, `est` 는 마스크에서 잔 추정 면적, `perim` 은 둘레임
 - 첫 줄은 표본마다 참 ÷ 추정을 구해 평균을 냄. 그것이 k 임

- 둘째 줄은 둘레에 비례하는 손실을 가정했을 때 어긋남이 가장 작아지는 $\hat{\epsilon}$ 를 구하는 식임. 어긋남의 제곱 합을 가장 작게 하는 계수를 고르는 방법을 **최소제곱**이라 하고, 이 꼴에서는 그 답이 $\Sigma(P \times (\text{참} - \text{추정})) \div \Sigma(P^2)$ 로 바로 나옴

□ 두 계수를 말로 옮기면

- $k = 1.128$ 은 "추정 면적에 1.128을 곱하라", 즉 12.8%를 더하라는 뜻임. 뒤집어 보면 추정 면적이 참값의 $1 \div 1.128 \approx 88.7\%$ 였다는 뜻임
- 마스크가 1,000m²라고 한 필지는 $1,000 \times 1.128 = 1,128\text{m}^2$ 로 고쳐 읽음. 모든 필지에 같은 12.8%를 더해 주는 방식이라 큰 땅과 작은 땅을 가리지 않음
- $\hat{\epsilon} = 0.916$ 은 "테두리 1m당 0.916m²가 사라졌다"는 뜻임. 둘레가 130m인 필지면 $0.916 \times 130 \approx 119\text{m}^2$ 를 더해 줌
- $\hat{\epsilon}$ 가 이론값 1에 가깝다는 것이, 이 자료에 심어 둔 오차가 실제로 한 겹 침식이라는 것을 뒷받침함

주의. 보정 계수는 표본을 어느 모집단에서 뽑았는지에 달려 있음. 같은 식으로 계산해도 실사 검토 풀 표본은 1.128, 검출된 필지 전수는 1.217, 검출된 모든 객체는 1.453(중앙값 1.162)이 나옴. 실사 풀 표본은 큰 필지 쪽으로 치우쳐 손실을 작게 보고, 모든 객체를 쓰면 과분할 조각(참/추정 ≈ 2)이 섞여 반대로 부풀림. 표본을 늘려도 사라지지 않으므로 "누구를 재서 계수를 만들었는가"를 보고서에 적음.

9.4 절단선을 비용으로 계산한다

□ 요건값을 임계로 그대로 쓸 이유는 없음

- **요건**은 사업이 원하는 땅의 기준이고, ****절단선(임계 τ)****은 오차가 섞인 추정값으로 그 기준을 거를 때 실제로 낫는 숫자임. 재는 값이 참값이 아니므로 둘이 같을 이유가 없음
- 후보 하나를 현장 확인하는 비용을 C_v , 요건을 만족하는 대상 하나를 놓치는 비용을 C_m 이라 하면 총비용은 다음과 같음

$$T(\tau)/C_v = N(\tau) + (C_m/C_v) \times M(\tau)$$

○ 식을 조각으로 읽으면

- $N(\tau)$ 는 임계 τ 에서 남은 후보 수, 즉 실제로 나가야 하는 방문 횟수임
- $M(\tau)$ 는 그 임계 때문에 떨어진 오탈락 수임
- 양변을 C_v 로 나눠 두었으므로 좌변은 "총비용이 실사 몇 건 값인가"로 읽음
- 우변 첫 항은 방문 횟수 그대로이고, 둘째 항은 놓친 대상을 실사 횟수로 환산한 값임

- **최적 τ 는 두 비용의 절대값이 아니라 비 C_m/C_v 에만 의존함.** "놓친 대상 하나가 실사 몇 건 값인가"만 답할 수 있으면 임계를 계산할 수 있다는 뜻임. C_v 가 100만 원이고 C_m 이 300만 원이면 식에 들어가는 값은 3임

- 요건이 여러 개면 후보는 갈때기를 차례로 통과함. 한 요건을 느슨하게 했을 때 늘어나는 후보 수는 그 요건이 단독으로 자르는 양이 아니라 **다른 요건을 통과한 것 중에서** 자르는 양임. 갈때기의 어느 칸이 실제로 좁은지 먼저 확인함

10. 사례 전체: 개발 가능 부지 탐색

- 도시 안의 유휴 부지를 찾아 현장 실사 대상을 좁히는 상황을 시뮬레이션함. 산출 단위가 객체(필지)이고 결정이 절단형이므로 7-8-9절의 계산이 모두 필요함
- 이 절의 표와 출력은 전부 7절 첫머리에서 실행한 `6-2-site-sourcing.py` 한 번의 결과이며, 로그 `lecture_practice/chapter6/results/6-2-site-sourcing.log` 의 [1] 부터 [11] 까지 구간에 그대로 찍혀 있음
- 아래 여섯 묶음은 읽는 순서가 정해져 있음. 앞의 결과가 뒤의 계산이 무엇을 할지를 정하기 때문임
 - 채점으로 **오차의 방향**을 확인하고 → 요건으로 걸러 **무엇을 잃었는지** 세고 → **보정으로 되돌려 보고** → 되돌아온 몫이 그 원인에서 왔는지 **대조군으로 확인하고** → 그래도 남는 오탈락은 **선 자체를 옮겨** 다름

○ 자료와 요건

```
# 출처: lecture_practice/chapter6/results/6-2-site-sourcing.log [1]
무대: 512x512 래스터(픽셀 1m=1㎡, 26.2ha) | 도로 50,005㎡ | 건물 26,877㎡
개발 요건(예시 설정): 면적 ≥ 1,000㎡, 접도 ≥ 12m, 최소폭 ≥ 15m

[1] 정답: 유휴 부지 32필지 중 요건을 모두 채우는 우량 부지 10필지
    탈락 사유 - 면적 미달 9 | 맹지·접도 부족 10 | 최소폭 미달 4 (중복 포함)
```

- 개발 요건은 예시로 설정한 사업 요건이며 법정 기준이 아님. 코드에는 `A_MIN · L_MIN · W_MIN` 세 상수로 적혀 있음
- 필지 면적은 최소 432㎡, 중앙값 1,155㎡, 최대 3,685㎡임(7절 첫머리의 준비 출력). 중앙값이 요건값 1,000㎡ 바로 위에 있다는 점이 중요함 — 절반 가까운 필지가 임계 근처에 몰려 있으므로 작은 면적 오차에도 판정이 뒤집힘
- 심어 둔 오차: 32필지 중 8필지는 두 겹 침식, 3필지(id 1-2-9)는 폭 2m 띠로 갈라 과분할, 산발 얼룩과 큰 오탐 덩어리 5개. 예측 마스크 총 면적은 55,313㎡로 정답의 **1.170배**
- **객체화와 채점**
 - 연결요소 949개 중 909개가 50㎡ 미만 얼룩 → 분석 대상 40개(7.2)
 - 필지 검출률 30/32(0.938), 오탐 객체 5개, 과분할 3필지·병합 0객체(8.1)
 - 면적 상대오차 중앙값 **-0.125**, 사분위수까지 전부 음수(8.2)
- **필터 갈때기와 오탈락**

출처: lecture_practice/chapter6/results/6-2-site-sourcing.log [4]

[4] 개발 요건 필터 깔때기 (면적 보정 전)

객체(얼록 제외)	40건
면적 $\geq 1,000\text{m}^2$	20건
접도 $\geq 12\text{m}$	9건
최소폭 $\geq 15\text{m}$	8건
후보 8건 우량 부지 오탈락 5건 (필지 [19, 20, 21, 22, 23])	

□ 화면에 나온 값에서 읽을 것

- 접도 판정의 허용오차는 3m를 씌(7.4). 허용오차를 0으로 두면 40개 중 37개가 탈락하고 남은 3개 중 진짜는 0개임
- 네 줄은 앞 줄을 통과한 것에만 다음 조건을 걸어 셈. 면적 조건이 40건을 20건으로(절반), 접도 조건이 20건을 9건으로(55% 탈락), 최소폭이 9건을 8건으로 자름
- **가장 좁은 칸은 접도임** — 한 요건을 느슨하게 했을 때 늘어나는 후보 수를 따지려면 이 순서를 먼저 봄(9.4). 면적 임계만 낮춰도 후보가 크게 늘지 않는 이유가 여기 있음
- 최종 후보 8건 중 우량 부지는 5필지뿐. **우량 부지 10필지 중 5필지(필지 19-20-21-22-23)가 목록에 오르지 못했고, 이들은 모두 침식으로 추정 면적이 임계 아래로 내려간 임계 근처 필지군임**
- **면적 편향 보정**
- 실사 검토 풀 16건 중 무작위 12건을 실측하고 2건은 부지가 아닌 것으로 판명되어 10건으로 계수를 추정함. 스칼라 보정 $k = 1.128$, 둘레 모형 $\hat{c} = 0.916$ (9.3)

출처: lecture_practice/chapter6/results/6-2-site-sourcing.log [6]

[6] 오탈락 분석 (이 예제의 결론표)

보정	후보 수	우량 부지 오탈락	우량 부지 포착	헛후보(비우량)
보정 없음	8	5	5	3
스칼라 k 보정	11	2	8	3
둘레 모형 보정	11	2	8	3

(전체 우량 부지 10필지 기준)

보정 뒤에도 남은 오탈락 [19, 21] - 그중 두 겹 침식 필지 2건. 전역 보정 계수 하나로는 필지마다 다른 침식 깊이를 되돌리지 못한다.

□ 화면에 나온 값에서 읽을 것

- 표를 읽는 법: 각 행의 「오탈락 + 포착」이 우량 부지 10필지를 나눈 것이고($5 + 5 = 10$, $2 + 8 = 10$), 「포착 + 헛후보」가 후보 수임($5 + 3 = 8$, $8 + 3 = 11$)
- 보정이 오탈락을 5건에서 2건으로 줄임. 되돌린 3건은 한 겹 침식으로 임계 바로 아래에 있던 필지이고, 남은 필지 19-21은 두 겹 침식이 걸린 필지임. **전역 계수 하나로는 필지마다 다른 침식 깊이를 되돌리지 못함**

- 12.8%를 더해 주는 보정으로는 11.3% 깎인 필지는 선 위로 올라오지만, 두 배로 깎인 필지는 여전히 선 아래에 남음
- 헛후보 3건은 세 조건 모두 동일함. **면적을 보정해도 유희지가 아닌 대상이 유희지가 되지는 않음.**
보정은 면적 숫자만 고치는 계산이고, 그 대상이 무엇인지는 고치지 않음
- 스칼라 보정과 둘레 모형 보정의 결과가 네 열 모두 같음. 두 방법의 차이를 가르려면 후보 수가 아니라 객체별 보정 면적을 봐야 함

○ 대조군

출처: lecture_practice/chapter6/results/6-2-site-sourcing.log [7]

[7] 대조군 - 심은 오차를 하나씩만 제거하고 같은 파이프라인을 돌린다

세계	검출 필지	면적오차	중앙값	후보 수	우량	오탈락	헛후보
본 실험(오차 3중)	30	-0.125	8	5	3		
C1 침식 제거	30	0.005	13	0	3		
C2 오탐 제거	30	-0.132	5	5	0		
C3 과분할 제거	32	-0.121	8	5	3		

□ 화면에 나온 값에서 읽을 것

- C1에서 면적오차가 -0.125 에서 0.005 로 사실상 사라지고 오탈락이 0이 됨 — **우량 부지를 농친 원인이 경계 침식이라는 주장이 이 조건에서 성립함.** 침식만 뺐더니 농침이 없어졌으므로 다른 원인을 더 찾을 필요가 없음
- C2에서 헛후보가 0이 됨 — 목록에 섞인 3건은 전부 오탐 덩어리였음. 다만 오탈락은 5건 그대로이므로, 오탐은 농침과는 무관함
- C3에서는 검출 필지가 30에서 32로 회복되지만 후보 수·오탈락·헛후보는 바뀌지 않음 — 과분할된 조각 하나가 이미 임계를 넘어 후보 자리를 지켰기 때문임. **채점 지표는 나빠지는데 결정은 바뀌지 않는 경우가 있다는 뜻임**

○ 절단선의 계산

출처: lecture_practice/chapter6/results/6-2-site-sourcing.log [8]

[8] 면적 임계를 어디에 둘 것인가 - 후보 수와 오탈락의 맞바꿈

농친 우량 부지 1건 = 실사 1건 비용일 때 → 최적 임계 1,450㎡ (후보 7건, 오탈락 5건)

농친 우량 부지 1건 = 실사 3건 비용일 때 → 최적 임계 725㎡ (후보 14건, 오탈락 0건)

농친 우량 부지 1건 = 실사 10건 비용일 때 → 최적 임계 725㎡ (후보 14건, 오탈락 0건)

참고: 요건 그대로인 1,000㎡에서는 후보 8건, 오탈락 5건

참고: 면적 편향(-0.125)만큼 낮춘 875㎡에서는 후보 11건, 오탈락 2건

□ 같은 값을 임계 순으로 다시 넣어놓으면 다음과 같음

임계 τ	후보 수	오탈락	비고
725m ²	14	0	농침이 실사 3배·10배 비용일 때 최적
875m ²	11	2	면적 편향(-0.125)만큼 낮춘 값
1,000m ²	8	5	요건값 그대로
1,450m ²	7	5	농침 1건 = 실사 1건 비용일 때 최적

- 농침 부지 1건이 실사 1건과 같은 값이면 최적 임계는 요건값보다 **높은** 1,450m²임. 어차피 농침 대상이라면 방문 횟수를 줄이는 쪽이 총비용을 낮추기 때문임
- 9.4의 식으로 확인하면, 1,000m²에서는 $8 + 1 \times 5 = 13$ 이고 1,450m²에서는 $7 + 1 \times 5 = 12$ 이므로 뒤쪽이 낮음. 오탈락이 5건으로 같은데 방문이 한 건 적음
- 농침이 실사보다 세 배 비싸면 725m²에서 $14 + 3 \times 0 = 14$ 이고 1,000m²에서는 $8 + 3 \times 5 = 23$ 이므로 임계를 내리는 쪽이 낮아짐. 오탈락이 0이 됨
- 접도·최소폭이 이미 대부분을 걸러 낸 뒤라 면적 임계를 크게 낮춰도 후보가 폭증하지 않음(8건→14건). 갈때기에서 가장 좁은 칸이 접도였던 것이 여기서 값으로 드러남
- 임계별 후보 수·오탈락·비용은 `ch6_threshold_cost.csv` 에 전 구간이 저장되므로 다른 비용비를 넣어 직접 다시 계산할 수 있음
- **실사 예산 6건일 때 방문 순서**

출처: lecture_practice/chapter6/results/6-2-site-sourcing.log [9]

[9] 실사 예산 6건 - 무엇을 먼저 볼 것인가

후보 11건 중 앞의 6건만 방문할 수 있다.

면적 큰 순 적중 4/6건(서로 다른 필지 4개) | 확보한 참 면적 10,166m²

여유폭 큰 순 적중 4/6건(서로 다른 필지 4개) | 확보한 참 면적 10,166m²

(참고) 정답을 아는 순 적중 6/6건(서로 다른 필지 6개) | 확보한 참 면적 11,413m²

헛걸음 2건의 정체 - 유희 부지가 아닌 오탈 2건, 부지이지만 요건 미달 0건

□ 화면에 나온 값에서 읽을 것

- 후보 11건은 보정을 적용한 뒤의 목록임(위 [6] 표의 11건)
- **여유폭 큰 순**은 세 요건 각각에 대해 "임계를 얼마나 여유 있게 넘겼는가"를 보고, 그중 가장 빠듯한 값이 큰 순서로 세운 것임. 어느 한 요건에서 간신히 넘긴 후보를 뒤로 미루는 순서임
- 면적 큰 순과 여유폭 큰 순의 성적이 $4/6건 \cdot 10,166m^2$ 로 완전히 같음. 정답을 아는 순의 $6/6건 \cdot 11,413m^2$ 가 도달 가능한 상한임
- 두 순서의 성적이 같은 이유는 헛걸음 2건이 모두 오탈 텅어리였고 요건 미달 부지는 0건이었기 때문임. 요건 미달이 섞여 있었다면 여유폭 순이 그것을 뒤로 밀어 성적이 같았을 것임
- 오탈 텅어리는 크고 넓고 도로에 붙어 있어 **기하 피쳐만으로는 우량 부지와 구별되지 않음**. 면적·접도·최소폭 세 값이 모두 우량 부지와 같은 범위에 있으므로, 어떤 순서로 세워도 앞쪽에 올라옴

- 오탐을 줄이려면 유희지와 포장 주차장을 가르는 정보(다중스펙트럼 반사 특성, 시계열 변화)를 모델 입력에 넣어야 하며 후처리로는 해결되지 않음. 순서를 바꿔 풀 문제가 아니라 분류를 고쳐야 하는 문제임

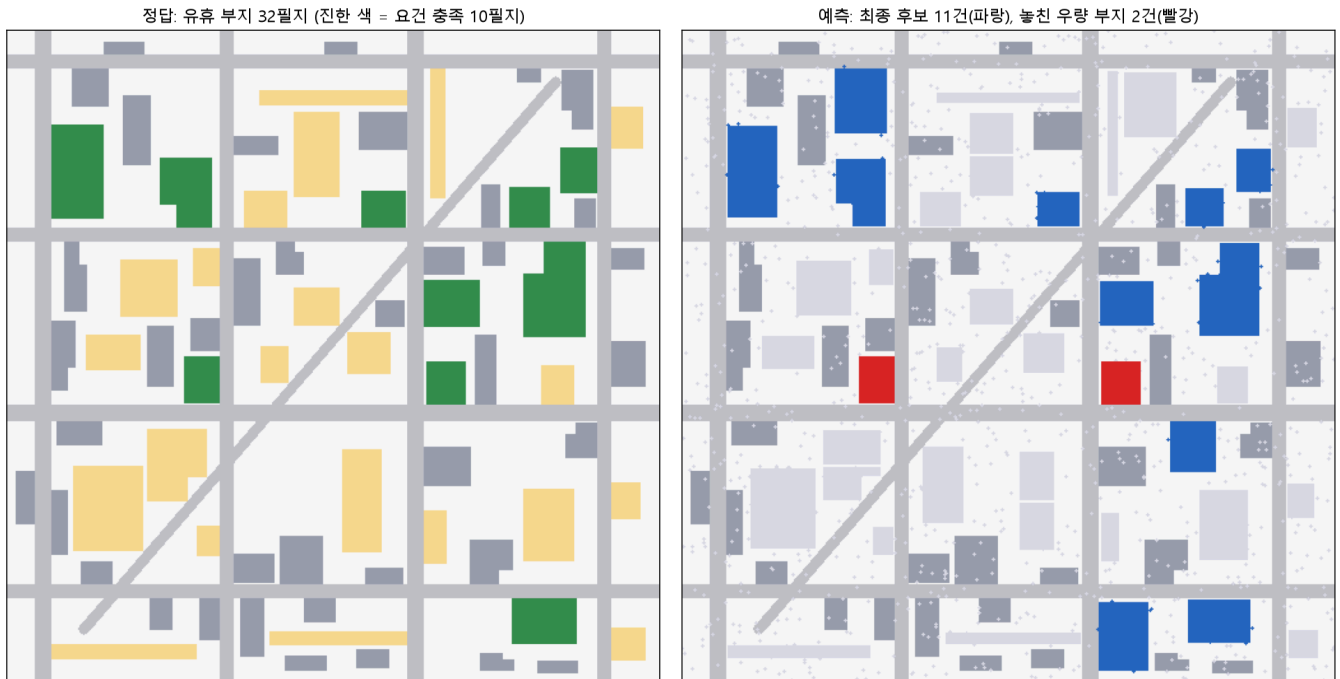


그림 6.6 개발 가능 부지 탐색의 정답과 결과. 왼쪽은 정답으로, 회색이 도로와 기존 건물, 옅은 색이 유희 부지 32필지, 진한 색이 세 요건을 모두 채우는 우량 부지 10필지임. 오른쪽은 예측 마스크에서 뽑은 최종 후보(파랑)와 우량 부지인데 목록에 오르지 못한 필지(빨강)임. 파란 후보 가운데 왼쪽에 대응하는 유희 부지가 없는 것이 유희지로 오인된 오탐 덩어리임

- 이 그림은 6-2-site-sourcing.py 를 실행해야 생성됨. 실행 전에는 파일이 없어 그림이 깨져 보임
- 이 사례가 뒷받침하는 것과 뒷받침하지 못하는 것
- 뒷받침하는 것: 면적 임계를 쓰는 후보 선별에서 세그멘테이션의 체계적 면적 편향이 임계 근처 객체를 통째로 탈락시킨다는 것, 소수 실측 표본에 의한 보정이 그 탈락의 일부를 되돌리지만 침식 깊이가 다른 대상은 되돌리지 못한다는 것, 기하 조건의 허용오차가 마스크의 위치 오차보다 작으면 필터가 뒤집힌다는 것
- 뒷받침하지 못하는 것: 실제 도시에서의 검출률과 면적 오차의 크기. 여기 심은 침식 폭·오탐률·과분할 빈도는 설계값이므로 12.5%라는 편향 크기를 다른 자료에 그대로 옮길 수 없음. **옮길 수 있는 것은 절차 자체임** — 편향의 방향과 크기를 소수 실측으로 재고 그 크기만큼 임계를 조정하는 절차
- 이 사례가 판정하는 것은 부지의 물리적 가용성뿐임. 용도지역, 건폐율·용적률, 소유관계, 토양오염 이력 같은 법적·행정적 제약은 다루지 않음. 이 목록은 실사 대상을 좁히는 1차 필터이지 개발 가능 판정이 아님

11. 핵심 요약

- **변화 탐지는 두 시점을 입력으로 받는 세그멘테이션임.** 계산의 뼈대는 같고, 정합이 0단계로 앞에 붙는 것과 정밀도·재현율의 우선순위가 결정 성격에 따라 뒤집히는 것이 다름 (3.1, 4.2)
- **산출 단위를 착수 시점에 정함.** 픽셀에서 끝나면 시맨틱으로 충분하고, 객체가 판단 단위이면 연결요소·기하 피쳐·객체 단위 채점이 모두 따라옴 (3.2)
- **대상의 픽셀 폭이 과제의 성패를 정함.** 경계 한 겹의 오차가 만드는 상대 면적 오차는 정사각형에서 4/한 변이므로, 한 변 32m 대상은 $4 \div 32 = 12.5\%$ 가 하한임 (4.1)
- **후처리 필터는 정밀도만 올림.** 예측 안의 가짜를 깎는 장치라서 모델이 놓친 픽셀은 되살릴 수 없음. 재현율의 식에 FP가 들어가지 않기 때문임. 실습에서 정밀도 0.572→1.000, 재현율 0.865 고정 (6.3)
- **불균형 자료에서 정확도는 다수 클래스의 비율을 보고할 뿐임.** 변화 1% 자료에서 전 픽셀을 배경이라 답해도 정확도 99%가 나옴. 그래서 IoU·정밀도·재현율은 TN을 세지 않음 (6.4)
- **연결요소의 개수는 대상의 개수가 아니고, 면적이 같아도 형상이 다르면 요건에서 걸림.** 실습에서 949개 중 909개가 얼룩이었고, 40×25 와 100×10 은 둘 다 1,000m²이지만 최소폭 요건에서 뒤쪽만 탈락함 (7.2, 7.3)
- **기하 조건의 허용오차는 마스크의 침식 깊이보다 크게 잡음.** 0으로 두면 침식된 진짜가 탈락하고 침식되지 않은 오탐만 살아남음. 실습에서 허용오차 0은 40개 중 37개를 떨어뜨리고 남은 3개에 진짜가 없었음 (7.4)
- **침식 오차는 방향이 있어 평균해도 남음.** 실습의 면적 상대오차 중앙값 -0.125이고 1.3사분위까지 전부 음수였으며, 작은 필지가 더 많이 깎임 (8.2)
- **원인 주장에는 대조군이 필요함.** C1에서 오탐락이 5건에서 0건이 되어야 "침식 때문에 놓쳤다"가 측정이 됨 (8.3, 10절)
- **절단형 결정에서는 오탐락이 산출물에 남지 않음.** 그래서 편향 보정과 절단선 계산이 추가로 필요하고, 최적 임계는 비용의 절대값이 아니라 비 C_m/C_v 가 정함 (9.2, 9.4)

12. 과제

```
python scripts/submit.py 6 --id 학번 --name 이름
```

- ☐ 실행하면 `submissions/` 폴더에 `제출_6장_학번.md`가 생성되고 `6-2-site-sourcing.py`의 실행 결과가 붙어 있음
 - **눈여겨볼 곳은 면적 임계별 후보 수와 오탐락 수.** 10절의 「절단선의 계산」표와 같은 값임
- 빈칸 세 개를 채움
 1. 개발 요건은 면적 1,000m² 이상이다. 임계를 그대로 1,000m²에 두겠는가, 낮추겠는가? 그 판단을 정하는 것은 무엇인가?

2. 보정 전후 표에서 오탈락이 5건에서 2건으로 줄었지만 0건이 되지는 않았다. 남은 2건에 공통으로 있는 것은 무엇이며, 왜 보정으로 되돌릴 수 없는가?
3. **실사 예산이 6건에서 12건으로 늘어난다면, 10절의 어느 표를 다시 보고 방문 순서를 정하겠는가? 그 표를 고른 이유를 적으라**

- **막혀도 제출함**
- 실행이 실패하면 오류 메시지가 파일에 담기고, 질문이 "어디서 막혔나"로 바뀜
- 실행 성공 여부로 점수를 매기지 않음

- **이 장의 두 실습은 시뮬레이션 자료를 씀**
- 분석 1은 저장소에 포함된 `change_masks.npz` 를 읽고, 분석 2는 저장소에 포함된 `urban_block.npz` 를 읽으므로 값이 사람마다 같아야 정상
- `FRONT_TOL` 같은 상수를 바꾸면 당연히 달라짐

부록. 이 장에서 자주 나오는 말

- 본문을 읽다 막힐 때 여기로 돌아오면 됨. 본문의 정의를 줄인 것이 아니라, 같은 말을 한 번 더 쉬운 문장으로 적어 둔 것임. 지리정보 전반의 용어는 `lecture/appendices.md` 의 부록 J(지리정보 기초와 용어 입문)에 있음

말	쉬운 뜻	본문
마스크	대상 칸만 1로 칠해 둔 격자. 영상과 크기가 같음	1절
픽셀	영상의 가장 작은 칸	1절
패치	모델이 한 번에 읽는 조각	1절
타일	영상을 저장·배포하는 큰 단위	1절
래스터	픽셀이 바둑판처럼 늘어선 격자형 영상	2절
세그멘테이션	칸마다 종류나 객체 번호를 붙이는 일	3.1
변화 탐지	두 시점 영상을 포개 어디가 바뀌었는지 칸마다 답하는 일	3.1
정합	두 영상을 같은 격자에 포개는 일	4.2
기하보정	영상을 좌표계에 맞춰 얹히는 처리	4.2
대기보정	센서 값에서 대기의 몫을 걷어 내 지표 반사율로 바꾸는 처리	4.3
거짓 변화	땅은 그대로인데 구름·계절·어긋남 때문에 변한 것처럼 보이는 값	4.3
풀링	2×2 칸 넷을 한 칸으로 줄이는 연산. 한 번에 해상도가 절반	5절
업샘플	한 칸을 2×2로 되돌려 해상도를 두 배로 늘리는 연산	5절
스킵 연결	줄어들기 전의 위치 정보를 디코더로 곧장 넘기는 통로	5절
경계 침식	마스크 가장자리가 실제보다 안쪽으로 깎이는 오차	5절, 8.2
NDVI	식생지수. 식생이 짙을수록 높고 -1~1 사이 값	6절
TP	진짜 변화를 맞힌 칸	6.1
FP	변화 아닌데 변화라고 한 칸	6.1
FN	진짜 변화인데 놓친 칸	6.1
TN	변화 아닌데 변화 아니라고 한 칸. 네 지표에 안 들어감	6.1, 6.4
정밀도	변화라고 한 것 중 진짜 비율. $TP \div (TP + FP)$	6.1
재현율	진짜 변화 중 잡아낸 비율. $TP \div (TP + FN)$	6.1
IoU	겹친 칸 ÷ 합친 칸. 0.5면 절반을 맞힌 것이 아니라 상당히 겹친 것	6.1
Dice	$2 \times \text{겹친 칸} \div (\text{예측 칸 수} + \text{정답 칸 수})$	6.1
임계값	모델이 낸 확률을 1로 찍을지 가르는 기준값	6.2
연결요소	맞당은 1 픽셀을 한 덩어리로 묶어 번호를 매기는 일	7.1

말	쉬운 뜻	본문
4-이웃 / 8-이웃	상하좌우만 이웃으로 보느냐, 대각선까지 보느냐	7.1
필지	등록된 땅 한 덩어리	7절
접도	땅의 경계 가운데 도로에 닿는 길이	7절, 7.3
맹지	도로에 닿는 경계가 없어 들어갈 길이 없는 땅	7절
크랙 둘레	1인 칸과 0인 칸 사이 경계선 토막의 길이를 모두 더한 값	7.3
조밀도	$4\pi A/P^2$. 원이 1, 정사각형이 약 0.785, 길쭉할수록 0에 가까움	7.3
거리변환	1인 칸마다 가장 가까운 0까지 몇 칸인지 적어 넣는 계산	7.3
최소폭	땅 안에 들어가는 가장 큰 원의 지름으로 대신 낸 폭	7.3
허용오차	도로에서 이 거리 안이면 접면으로 인정하는 여유값. 여기서는 3m	7.4
과분할	한 필지를 여러 조각으로 쪼개는 오차	8.1
병합	여러 필지를 한 덩어리로 붙이는 오차	8.1
면적 상대오차	$(\text{추정 면적} - \text{참 면적}) \div \text{참 면적}$. 음수면 작게 뺌	8.2
사분위수	값을 크기순으로 줄 세워 넷으로 나눈 경계값	8.2
오탐	대상이 아닌데 대상으로 잡은 덩어리	8.1, 10절
대조군	다른 조건은 그대로 두고 한 원인만 제거해 다시 돌린 조건	8.3
정렬형	큰 것부터 순서를 매겨 자원을 나누는 결정	9.2
절단형	선을 넘겨야 목록에 남는 결정. 떨어진 이름은 표에 안 남음	9.2
오탈락	요건을 실제로 만족하는데 추정값이 선 아래라 빠진 것	9.2
헛후보	요건을 만족하지 않는데 목록에 올라온 것	9.2
스칼라 보정	추정 면적에 같은 배율 k 를 곱해 키우는 보정	9.3
둘레 모형	추정 면적에 $\hat{c} \times \text{둘레}$ 를 더하는 보정	9.3
최소제공	어긋남의 제공 합이 가장 작아지는 계수를 고르는 방법	9.3
요건	사업이 원하는 통과 조건. 예: 면적 1,000m ²	9.4
절단선(임계 τ)	오차가 있는 추정값으로 실제로 자르는 숫자. 요건과 다를 수 있음	9.4
Cm/Cv	놓친 대상 1건이 현장 확인 몇 건 값인가	9.4

1. **픽셀, 패치, 타일의 공간적 위계**: 위성영상 분석에서 **타일(Tile)**은 원본 영상의 배포 및 저장 단위(예: 10000x10000)를, **패치(Patch)**는 AI 모델이 한 번에 처리할 수 있도록 잘라낸 조각(예: 256x256)을, **픽셀(Pixel)**은 영상의 가장 작은 점 하나(최소 데이터 단위)를 의미합니다. 타일 $\supset$ 패치 $\supset$ 픽셀의 포함 관계를 가집니다.↩
2. Law, S., Paige, B., Russell, C. (2019). Take a Look Around: Using Street View and Satellite Images to Estimate House Prices. *ACM TIST* 10(5). 위 표의 수치는 채택본 PDF(https://www.turing.ac.uk/sites/default/files/2019-07/1807.07155_1_0.pdf) 표 2·표 3의 R^2 값임. **이 장의 다른 수치와 달리 직접 실행해 얻은 값이 아니라 외부 연구에서 가져온 값임.** 그 논문의 기준 모형 변수는 거래연도·면적·방 수·건물나이·유형·공원까지 거리·800m 내 상점 수·고용 접근성 여덟 개뿐이고, 전문에 spatial lag-spatial autocorrelation-comparable이 한 번도 나오지 않음.↩